254374 System Analysis and Design: สรุปเปิดหนังสือ ปลายภาค

สอบ 5 ข้อ เขียนบรรยายและคำนวณ 3 ชั่วโมง เปิดหนังสือ ใช้เครื่องคิดเลขได้ เนื้อหาหลังสอบกลางภาค

ดัชนี: โจทย์ถามแบบไหน ไปอ่านที่ไหน

โจทย์ถามเรื่อง	ส่วน	หน้า
หาเส้นทางวิกฤต เวลาโครงการต่ำสุด slack ES/EF/LS/LF	1.4 เส้นทางวิกฤตและ Slack	3
อัดฉีดเงิน (crash) ลดเวลา ต้นทุนรวมใหม่ ตารางเส้นทาง	1.5 Crash	4
แบบฝึกหัด HW04 (activity-on-arrow, dummy)	1.6 HW04	6
Feasibility 3 ด้าน งานบริหารโครงการ 4 งาน Gantt vs PERT	1.1 ถึง 1.3	2
ระบบฐานข้อมูล DBMS ข้อดี องค์ประกอบ schema/subschema	2.1 DBMS	8
Key (PK, candidate, FK, secondary), referential integrity	2.2 ศัพท์ข้อมูลและ Key	9
วาด ERD, 1:1 1:M M:N, crow's foot cardinality	2.3 ERD	9
Normalization UNF→3NF แยกตาราง ERD หลัง 3NF (คลินิกฟัน)	2.4 Normalization	10
Data warehouse / mart / mining, data control, ออกแบบตารางจากโจทย์	2.5 Warehouse	11
SaaS, Web 2.0, Cloud, Web-based vs Traditional	3.1 Internet	13
Build vs Buy, outsourcing, horizontal/vertical, ค่าใช้จ่าย outsource	3.2 ทางเลือกพัฒนา	13
Payback period (ไม่มีอัตราคิดลด)	3.4 Payback	14
PV, PV factor, อ่านตาราง	3.5 PV	15
Discounted payback (มีอัตราคิดลด ถามกี่ปี)	3.6 Discounted payback	15
NPV ควรลงทุนหรือไม่ เลือกโครงการ	3.7 NPV	16
ประเภท output, รายงาน 3 ชนิด ส่วนประกอบรายงาน	4.2 รายงาน	19
คำนวณจำนวนหน้า/แผ่น/เวลาพิมพ์รายงาน	4.3 คำนวณรายงาน	20
User rights 10 ข้อ, UI guidelines 8 ข้อ, controls	4.4 UI	20
Input, batch vs online, ลด input, validation 8 แบบ, audit trail	4.5 Input	21
Unit / integration / system test	5.1 ทดสอบ	24
อบรม 3 กลุ่ม, แปลงข้อมูล	5.3 และ 5.4	24
Changeover 4 วิธี เสี่ยง/ค่าใช้จ่าย	5.5 Changeover	25
Maintenance 4 ประเภท, help desk, final report	5.7 Maintenance	25
เกร็งข้อสอบ: รูปแบบโจทย์ → ตอบอะไร	ท้ายแต่ละบท (1.7, 2.6, 3.8, 4.6, 5.8)	7
สูตรทั้งหมด ตาราง PV factor	ภาคผนวก A	27

ลำดับคิดตอนทำข้อสอบ

1. ข้อคำนวณ: ตั้งตารางแบบเดียวกับตัวอย่างในสรุป เขียนชื่อ = สูตร = แทนตัวเลข = ผล
2. ปิดท้ายทุกข้อคำนวณด้วยหน่วย และประโยคคำตอบ/คำแนะนำ (เช่น ควรลงทุน เพราะ NPV > 0)
3. ข้อบรรยาย ตอบเป็นรายการตามที่สไลด์แจกแจง (เช่น 8 validation checks, 4 changeover) ให้ครบทุกข้อ แล้วใส่ตัวอย่างสั้น ๆ

1 วางแผนโครงการ Project Planning, PERT/CPM, Crash

1.1 ทำไมสร้างระบบ และต้องศึกษาอะไรก่อน objectives, feasibility

ศัพท์	ความหมาย
วัตถุประสงค์สร้าง/ปรับปรุงระบบ 5 ข้อ	Improve Service ปรับปรุงบริการลูกค้า/ผู้ใช้ให้ดีขึ้น; Support for new products and services รองรับบริการใหม่; Better performance ทำงานมีประสิทธิภาพขึ้น; Stronger control ควบคุมได้ดีขึ้น; Reduced cost ลดต้นทุน
ความเป็นไปได้ 3 ด้าน Feasibility	ต้องศึกษาก่อนลงมือสร้างระบบ
Technical feasibility	เทคนิค: เทคโนโลยี ฮาร์ดแวร์ ทักษะทีมที่มี สร้างระบบนี้ได้จริงไหม
Economic feasibility	เศรษฐศาสตร์: ผลตอบแทนคุ้มกับต้นทุนไหม (คำนวณด้วย Payback / NPV ในบทที่ 3)
Schedule feasibility	เวลา: ทำเสร็จทันกำหนดไหม (ใช้ตารางเวลา Gantt / PERT)

หลังศึกษาความเป็นไปได้แล้ว ผู้พัฒนาต้องทำต่อ 5 อย่าง: กำหนดขอบเขตงาน **project scope**; กำหนดวัตถุประสงค์หลัก **project goal**; ระบุปัญหาของระบบปัจจุบัน **problem identification**; ประมาณต้นทุน **cost estimation**; ระบุทรัพยากรที่ต้องใช้ **identify required resources**.

1.2 การบริหารโครงการและการแบ่งงาน project management, task decomposition

ศัพท์	ความหมาย
งานหลักของการบริหารโครงการ 4 งาน Project management	1) Project planning วางแผน 2) Project scheduling กำหนดตารางเวลา 3) Monitoring and Controlling ตรวจสอบและควบคุม 4) Reporting รายงานความก้าวหน้าให้ทุกฝ่ายที่เกี่ยวข้อง
การแบ่งงาน Task decomposition	วางแผนว่าระบบประกอบด้วยงานย่อยอะไรบ้าง: มองภาพรวมแล้วแตกลงเป็นงานย่อยเรื่อย ๆ แสดงเป็นโครงสร้างลำดับชั้น (Hierarchical structure) เช่น สร้างบ้าน แตกเป็น สาธารณูปโภค (ไฟฟ้า น้ำประปา), วัสดุ (กระเบื้อง หลังคา ปูน), ตกแต่ง (เฟอร์นิเจอร์ ผ้าม่าน วอลเปเปอร์)
การวางแผนเวลา Time planning	กำหนดตารางเวลาของงานหลักทั้งหมด; ต้อง **ประมาณเวลาของแต่ละงานให้ถูกก่อน** จึงสร้างตารางเวลาได้

1.3 เครื่องมือตารางเวลา: Gantt และ PERT/CPM

	Gantt chart	PERT / CPM chart
หน้าตา	แผนภูมิแท่งแนวนอน: แถว = งาน, แกนนอน = เวลา	กราฟเครือข่าย: โหนด/ลูกศร = งาน และการเชื่อมต่อ
เห็นอะไรชัด	วันเริ่ม วันจบ ระยะเวลา และงานที่ซ้อนกันในช่วงเวลา	ความขึ้นต่อกันของงาน (ใครต้องเสร็จก่อนใคร), เส้นทางวิกฤต, slack
ใช้เมื่อ	ดูตารางเวลา/ความคืบหน้า	หาเส้นทางวิกฤตและคำนวณ crash

ศัพท์	ความหมาย
PERT Program Evaluation Review Technique	แสดงสายงานที่เชื่อมต่อกันเป็นกราฟ
ข้อมูลของแต่ละโหนด (งาน) node data	Task Name ชื่องาน; Task ID รหัสงาน; Task Duration ระยะเวลา; Start Day/Date วันเริ่ม; Finish Day/Date วันเสร็จ
งานที่ขึ้นต่อกัน Dependent task	งานที่จะเสร็จได้ต้องรองานอื่นเสร็จก่อน
Multiple Successor tasks concurrent tasks	งานที่เริ่มพร้อมกันได้ (ขึ้นกับงานก่อนหน้าอันเดียวกัน) เช่น งาน 2 และ 3 ต่างตามหลังงาน 1
Multiple Predecessor tasks	งานที่เริ่มได้ต้องรองานก่อนหน้ามากกว่า 1 งาน: **เริ่มหลังงานที่เสร็จช้าที่สุดเสร็จ** (เช่น งาน 10 ต้องรอทั้ง 7 และ 8)
เส้นทางวิกฤต Critical path	เส้นทางที่ถ้างานใดบนเส้นล่าช้า โครงการจะล่าช้าตาม = **เส้นทางที่ใช้เวลารวมมากที่สุด** ใน PERT
เวลาหย่อน Slack time	เวลาที่งานหนึ่งเสร็จช้ากว่ากำหนดได้โดยไม่ทำให้โครงการเสร็จช้า; งานบนเส้นวิกฤตมี slack = 0
Dummy activity	งานสมมติ ใช้เวลา 0 และต้นทุน 0 วาดเพื่อแสดงความขึ้นต่อกัน (HW04: งาน 3→4) crash ไม่ได้
CPM Critical Path Method (สไลด์เขียน Management)	ใช้เงินเข้ามาช่วยลดเวลางานบนเส้นวิกฤต เช่น จ้างคนเพิ่ม งานเสร็จเร็วขึ้นแต่ต้นทุนสูงขึ้น เรียกว่า Crash (อัดฉีดเงิน)

1.4 คำนวณ: เส้นทางวิกฤตและ Slack critical path, slack

ใช้เมื่อ โจทย์ให้ตารางงาน (ระยะเวลา + งานก่อนหน้า) แล้วถามเวลาโครงการต่ำสุด เส้นทางวิกฤต หรือ slack ของงาน.

วิธีที่ 1: ไล่ทุกเส้นทาง (เร็ว เหมาะข้อสอบ)

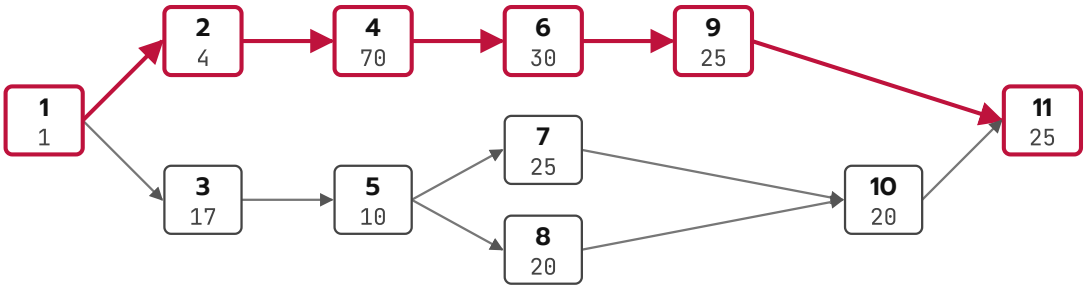
1. วาด PERT: งานที่ไม่มีงานก่อนหน้า (-) เป็นจุดเริ่ม; ลูกศรจากงานก่อนหน้าไปงานที่ตามมา
2. ไล่ทุกเส้นทางจากจุดเริ่มถึงจุดสุดท้าย บวกระยะเวลา เช่น `A+B+D`
3. เส้นทางที่ **ยาวที่สุด** = critical path; เวลานั้น = เวลาโครงการต่ำสุด (ใส่ * ที่เส้นวิกฤต)
4. งานบนเส้นวิกฤต slack = 0; งานอื่น slack = (ความยาวเส้นวิกฤต) - (ความยาวเส้นที่ยาวที่สุดที่งานนั้นอยู่)

วิธีที่ 2: Forward / Backward pass (ใช้เมื่อถาม ES EF LS LF)

> **EF = ES + D** ES = max EF ของงานก่อนหน้า
> **LS = LF - D** LF = min LS ของงานที่ตามมา
> **Slack = LS - ES = LF - EF**

สัญลักษณ์	ความหมาย	หาจากไหน
`D`	Duration ระยะเวลางาน	จากตารางโจทย์
`ES`	Earliest Start เริ่มเร็วสุด	งานแรก = 0; งานอื่น = EF ที่มากสุดของงานก่อนหน้า (รอตัวช้าสุด)
`EF`	Earliest Finish เสร็จเร็วสุด	ES + D (forward pass: ไล่ซ้ายไปขวา)
`LF`	Latest Finish เสร็จช้าสุดที่ยังไม่ทำให้โครงการช้า	งานสุดท้าย = EF ของมัน (= เวลาโครงการ); งานอื่น = LS ที่น้อยสุดของงานที่ตามมา
`LS`	Latest Start เริ่มช้าสุด	LF - D (backward pass: ไล่ขวาไปซ้าย)
`Slack`	เวลาหย่อน	LS - ES; = 0 แปลว่างานอยู่บนเส้นวิกฤต

ตัวอย่างจากสไลด์ (งาน 11 งาน หน่วยวัน): โครงข่าย ตารางผล และเส้นทาง



งาน	ชื่อ	D	ก่อนหน้า	ES	EF	LS	LF	Slack
1*	วางแผน	1	-	0	1	0	1	0
2*	กำหนดงานต่าง ๆ	4	1	1	5	1	5	0
3	จัดซื้ออุปกรณ์	17	1	1	18	58	75	57
4*	เขียนโปรแกรม	70	2	5	75	5	75	0
5	ติดตั้งอุปกรณ์	10	3	18	28	75	85	57
6*	ทดสอบโปรแกรม	30	4	75	105	75	105	0
7	เขียนคู่มือ	25	5	28	53	85	110	57
8	เปลี่ยนรูปแบบไฟล์	20	5	28	48	90	110	62
9*	ทดสอบระบบ	25	6	105	130	105	130	0
10	อบรมการใช้งาน	20	7, 8	53	73	110	130	57
11*	ทดสอบโดยผู้ใช้จริง	25	9, 10	130	155	130	155	0

เส้นทาง	คำนวณ	รวม (วัน)
1-2-4-6-9-11 *	1+4+70+30+25+25	155 (วิกฤต)
1-3-5-7-10-11	1+17+10+25+20+25	98
1-3-5-8-10-11	1+17+10+20+20+25	93

ตอบ: โครงการใช้เวลาต่ำสุด **155 วัน**, critical path = 1, 2, 4, 6, 9, 11 (slack 0). Slack งาน 3 = 155 - 98 = 57; งาน 8 = 155 - 93 = 62. งาน 10 ต้องรอทั้ง 7 (EF 53) และ 8 (EF 48) จึง ES = 53.

1.5 คำนวณ: Crash (อัดฉีดเงินลดเวลา) แบบ CPM crash cost per day, crash table

ใช้เมื่อ โจทย์ให้ Normal time/cost และ Crash time/cost แล้วถามว่าลดเวลาโครงการได้กี่วัน ใช้เงินเท่าไร.

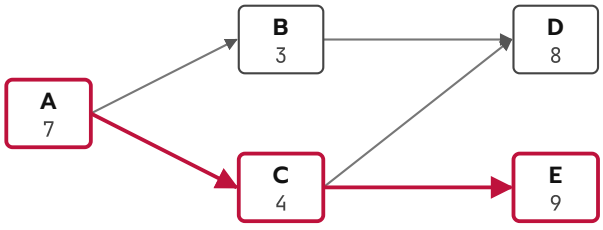
> ลดได้สูงสุด $\mathbf{m} = \mathbf{Tn} - \mathbf{Tc}$
>
> ต้นทุน/วัน $\mathbf{k} = \dfrac{\mathbf{Cc} - \mathbf{Cn}}{\mathbf{Tn} - \mathbf{Tc}}$ ต้นทุนรวมหลังขั้นนี้ $\mathbf{Total} = \text{ก่อนหน้า} + \Sigma(\mathbf{วันที่ลด} \times \mathbf{k})$

สัญลักษณ์	ความหมาย	หาจากไหน
`Tn`	Normal duration เวลาปกติ	คอลัมน์ Normal Duration
`Tc`	Crash duration เวลาเมื่ออัดฉีดเต็มที่	คอลัมน์ Crash Duration
`Cn`	Normal cost ต้นทุนปกติ	คอลัมน์ Normal Cost
`Cc`	Crash cost ต้นทุนเมื่ออัดฉีดเต็มที่	คอลัมน์ Crash Cost (เป็นต้นทุนรวมของงาน ไม่ใช่ส่วนเพิ่ม)
`m`	จำนวนวันที่ลดได้สูงสุดของงานนั้น	Tn - Tc; crash เกินนี้ไม่ได้ (ใช้ได้หลายครั้งจนครบ m)
`k`	ต้นทุนอัดฉีดต่อ 1 วัน	(Cc - Cn) / (Tn - Tc)
`Total`	ต้นทุนรวมโครงการ	เริ่มต้น = ผลบวก Normal cost ของ **ทุกงาน** (รวมงานนอกเส้นวิกฤต) แล้วบวกเงินอัดฉีดทีละขั้น

ขั้นตอน

1. คำนวณ `m` และ `k` ของทุกงาน (ตาราง); ต้นทุนเริ่มต้น = ผลรวม Normal cost
2. ไล่ทุกเส้นทางและเวลารวม; ใส่ * ที่เส้นที่ยาวที่สุด (ถ้ายาวเท่ากันหลายเส้นก็ * ทุกเส้น)
3. เลือกงานที่จะ crash: ต้อง **อยู่บนเส้นวิกฤต** ยังเหลือ m และ **ลดแล้วทุกเส้นวิกฤตสั้นลงพร้อมกัน**; ถ้ามีหลายเส้นวิกฤต ให้เลือกงานร่วม (อยู่ในทุกเส้น) ที่ k ต่ำสุด หรือจับคู่หลายงาน (เช่น B+C) ให้ครอบคลุมทุกเส้น
4. ลดกี่วัน = ค่าน้อยกว่าระหว่าง (m ที่เหลือ) กับ (ช่องว่างระหว่างเส้นวิกฤตกับเส้นที่ยาวรองลงมา) ไม่งั้นเส้นวิกฤตเปลี่ยนกลางทาง (สไลด์: E ลดได้ 3 แต่ลดแค่ 1 เพราะ ACD = 19 รออยู่)
5. อัปเดตตาราง: ลบวันออกจากทุกเส้นที่มีงานนั้น; ต้นทุนรวม += วัน × k; ทำเครื่องหมาย * ใหม่
6. หยุดเมื่อ งบเต็ม / ไม่มีงานให้ crash ต่อ (m หมด) / ถึงเป้าเวลาที่โจทย์กำหนด; ตอบเวลาใหม่ ต้นทุนรวม และส่วนต่างจากเดิม

ตัวอย่างจากสไลด์ งาน A-E: โครงข่าย ข้อมูล m และ k



งาน	ก่อนหน้า	Tn	Cn	Tc	Cc	m = Tn-Tc	k = (Cc-Cn)/m
A	-	7	3,000	4	6,000	3	(6,000-3,000)/3 = 1,000
B	A	3	4,000	2	5,500	1	(5,500-4,000)/1 = 1,500
C	A	4	15,000	2	20,000	2	(20,000-15,000)/2 = 2,500
D	B, C	8	10,000	5	19,000	3	(19,000-10,000)/3 = 3,000
E	C	9	7,000	6	9,100	3	(9,100-7,000)/3 = 700
รวม Normal			**39,000**				

เส้นทาง: ABD = 7+3+8 = 18, ACD = 7+4+8 = 19, ACE = 7+4+9 = 20 (วิกฤต). ตารางตอบ (แต่ละคอลัมน์ = 1 ขั้นที่ลดโครงการ):

เส้นทาง	เริ่มต้น	E ลด 1	A ลด 3	C ลด 1	B 1 + C 1	D 1 + E 1
ABD	18	18	15	15*	14*	13*
ACD	19	19*	16*	15*	14*	13*
ACE	20*	19*	16*	15*	14*	13*
คิดเงินขั้นนี้		1×700	3×1,000	1×2,500	1,500+2,500	3,000+700
ต้นทุนรวม	**39,000**	**39,700**	**42,700**	**45,200**	**49,200**	**52,900**

- ขั้น E: อยู่บนเส้นวิกฤต ACE และถูกสุด (700) ลดได้ 1 วัน แล้ว ACD เป็นวิกฤตร่วม (19).

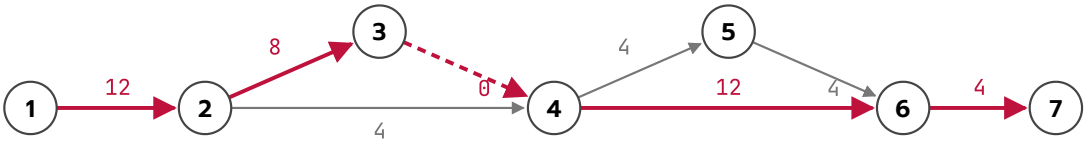
- ขั้น A: อยู่ในทุกเส้น และ k = 1,000 ต่ำสุดในงานร่วม จึงลดเต็ม 3 วัน (ABD = 15 ยังไม่เกิน 16).
- ขั้น C: อยู่ใน ACD และ ACE (2,500); ลด 1 วัน (ABD = 15 พอดี ทุกเส้นเท่ากัน).
- ขั้น B+C: ทั้ง 3 เส้นวิกฤต (15) ต้องลดทุกเส้น; C ครอบ ACD+ACE, B ครอบ ABD = 4,000. ลด B เดี่ยว ๆ ไม่ทำให้โครงการสั้นลง.
- ขั้น D+E: D ครอบ ABD+ACD, E ครอบ ACE = 3,700; งบจำกัด 55,000 จึงหยุดที่นี่.
- **ตอบ** โครงการลดเหลือ **13 วัน** (จาก 20), ต้นทุนรวม **52,900** บาท, เพิ่มขึ้น 52,900 - 39,000 = **13,900** บาท.

ทางเลือกที่ถูกกว่า (ไม่ใช่ลำดับของอาจารย์): ที่ 15 วัน ทั้ง 3 เส้นวิกฤต เทียบต้นทุน/วันของแต่ละชุดที่ตัดได้ทุกเส้น: B+C = 4,000, D+E = 3,000+700 = 3,700 (D ตัด ABD+ACD, E ตัด ACE) → ทำ D+E สองรอบ ได้ 14 วัน 48,900 แล้ว 13 วัน **52,600**; B+C อีกรอบ = 12 วัน 56,600 (เกินงบ 55,000 จึงหยุดที่ 13 วัน). **กฎ**: เมื่อมีหลายเส้นวิกฤต ให้เทียบต้นทุน/วันของทุกชุดที่ตัดได้ทุกเส้นพร้อมกัน แล้วเลือกชุดที่ถูกที่สุด.

1.6 ตัวอย่างทำเต็ม: HW04 แบบฝึกหัด (หน่วยสัปดาห์) activity-on-arrow, dummy

โจทย์: 1) ต้นทุนอัดฉีดต่อสัปดาห์และจำนวนครั้งสูงสุด 2) วาด PERT 3) ทุกเส้นทางและเส้นวิกฤต 4) crash ใช้เงินเท่าไร ลดได้กี่สัปดาห์. (ตัวเลขบนลูกศร = Normal duration; เส้นประ = dummy)

ขั้นตอนทั่วไปแบบ activity-on-arrow: ลูกศร = งาน, วงกลมเลข = เหตุการณ์; (1) คำนวณ m และ k ทุกลูกศร (2) ไล่ทุกเส้นทางจากเหตุการณ์แรกถึงสุดท้าย เส้นยาวสุด = วิกฤต (3) เลือกลูกศรบนเส้นวิกฤตที่ k ต่ำสุด ลดได้ไม่เกิน m และไม่ให้เส้นอื่นกลายเป็นยาวกว่า (4) หักวันจากทุกเส้นที่ผ่านลูกศรนั้น บวกเงิน วัน×k (5) ซ้ำจน m ของงานบนเส้นวิกฤตหมด. ผลเฉลย HW04: 36 → 24 สัปดาห์, 75,000 → 106,500 (+31,500) โดย crash 1→2 ลด 5 (400/สัปดาห์), 2→3 ลด 3 (500), 6→7 ลด 1 (7,000), 4→6 ลด 3 (7,000).



งาน	ก่อนหน้า	Tn	Cn	Tc	Cc	m	k
1→2	-	12	3,000	7	5,000	5	(5,000-3,000)/5 = 400
2→3	1→2	8	2,000	5	3,500	3	(3,500-2,000)/3 = 500
2→4	1→2	4	4,000	3	7,000	1	(7,000-4,000)/1 = 3,000
3→4 (dummy)	2→3	0	0	0	0	0	0
4→5	2→4, 3→4	4	500	1	1,100	3	(1,100-500)/3 = 200
4→6	2→4, 3→4	12	50,000	9	71,000	3	(71,000-50,000)/3 = 7,000
5→6	4→5	4	500	1	1,100	3	200
6→7	4→6, 5→6	4	15,000	3	22,000	1	(22,000-15,000)/1 = 7,000
รวม Normal			**75,000**				

เส้นทาง	เริ่มต้น	1→2 ลด 5	2→3 ลด 3	6→7 ลด 1	4→6 ลด 3
1-2-3-4-5-6-7	32	27	24	23	23
1-2-3-4-6-7	36*	31*	28*	27*	24*
1-2-4-5-6-7	28	23	23	22	22
1-2-4-6-7	32	27	27	26	23
คิดเงินขั้นนี้		5×400	3×500	1×7,000	3×7,000
ต้นทุนรวม	**75,000**	**77,000**	**78,500**	**85,500**	**106,500**

ลำดับตามอาจารย์: 1→2 (400 อยู่ทุกเส้น) → 2→3 (500 อยู่บนเส้นวิกฤต) → 6→7 (7,000 อยู่ทุกเส้น ลดได้ 1) → 4→6 (7,000 ลด 3). หลังขั้นสุดท้ายงานบนเส้นวิกฤตเหลือ m = 0 ทุกงาน จึงหยุด. **ตอบ** เส้นวิกฤตเดิม 1-2-3-4-6-7 = 12+8+0+12+4 = 36 สัปดาห์; หลัง crash เหลือ **24 สัปดาห์** (ลด 12), ต้นทุนรวม **106,500** บาท (เพิ่ม 31,500).

เกร็งข้อสอบ

โจทย์ถามแบบนี้	ตอบ / เลือกแบบนี้
หา critical path จากตารางงาน	ไล่ทุกเส้น บวกเวลา เส้น **ยาวที่สุด** = วิกฤต (ไม่ใช่สั้นสุด); slack ของงานบนเส้น = 0
งานมี 2 งานก่อนหน้า (เช่น 7, 8 → 10)	เริ่มหลังงานที่เสร็จ **ช้ากว่า**: ES = max(EF ก่อนหน้า)
slack ของงานนอกเส้นวิกฤต	= เวลาโครงการ - ความยาวเส้นที่ยาวที่สุดที่ผ่านงานนั้น (หรือ LS - ES)
ต้นทุนอัดฉีด/วัน	(Crash cost - Normal cost) / (Normal time - Crash time) ใช้ต้นทุนรวมของงานทั้งสองค่า ไม่ใช่ส่วนเพิ่ม
จำนวนครั้งสูงสุดที่อัดฉีดได้	= Normal time - Crash time (วัน/สัปดาห์)
ควร crash งานไหน	งาน **บนเส้นวิกฤต** ที่ k ต่ำสุด (ไม่ใช่ k ต่ำสุดทั้งตาราง เช่น B ไม่อยู่บน ACE จึงไม่อัดฉีด)
มีหลายเส้นวิกฤตเท่ากัน	ขั้นหนึ่งต้องลดให้ **ทุกเส้น**: crash งานร่วม หรือจับคู่หลายงาน; crash เส้นเดียวแล้วโครงการไม่สั้นลง
ต้นทุนรวมหลัง crash	Normal cost **ทุกงาน** + Σ(วัน×k)
ลดได้กี่วัน / ใช้เงินเท่าไร	ตอบ 3 ค่า: เวลาใหม่, ต้นทุนรวมใหม่, ส่วนต่าง (เช่น 20 → 13 วัน, 52,900, +13,900)
ข้อผิดในสไลด์ crash	สไลด์พิมพ์ต้นทุนหลัง D ลด 1 เป็น 49,200 ที่ถูก = 52,200 (49,200+3,000); ผลสุดท้าย 52,900 ถูกต้อง. ขั้น 15 → 14 ชุด D+E = 3,700 ถูกกว่า B+C = 4,000 (จะได้ 13 วันที่ 52,600) แต่คำตอบอาจารย์คือ 52,900: ตอบตามวิธีสไลด์ และเขียนวิธีคิดกำกับ
เทียบ Gantt กับ PERT	Gantt = แท่งเวลา เห็นช่วงเวลา; PERT = เครือข่าย เห็นความขึ้นต่อกัน/เส้นวิกฤต
ความเป็นไปได้ 3 ด้าน	Technical (ทำได้ไหม), Economic (คุ้มไหม), Schedule (ทันไหม)

2 ออกแบบฐานข้อมูล Data Design, ERD, Normalization

2.1 โครงสร้างข้อมูล ไฟล์ vs ฐานข้อมูล และ DBMS data structure, file vs database, DBMS

ศัพท์	ความหมาย
โครงสร้างข้อมูล Data structure	กรอบสำหรับจัดระเบียบ เก็บ และจัดการข้อมูล ประกอบด้วยไฟล์/ตารางที่เชื่อมโยงกัน; แต่ละไฟล์/ตารางเก็บข้อมูลของคน สถานที่ สิ่งของ หรือเหตุการณ์
การประมวลผลแบบไฟล์ File processing	แต่ละระบบมีไฟล์ของตัวเอง ข้อมูลเดียวกันถูกกรอกซ้ำหลายที่ (redundancy) ไม่มีประสิทธิภาพและผิดพลาดง่าย (ร้าน Mario: 2 ระบบแยกกัน). ยังใช้อยู่เมื่อต้องประมวลผลข้อมูลปริมาณมากที่มีโครงสร้างตายตัวเป็นประจำและคุ้มค่า เช่น บริษัทบัตรเครดิตบันทึกธุรกรรมนับพันรายการต่อวัน
ฐานข้อมูล Database	รวมข้อมูลไว้ที่เดียวไม่ซ้ำซ้อน มองเหมือนตารางใหญ่ตารางเดียวไม่ว่าเก็บจริงที่ไหน (ร้าน Danica); ฐานข้อมูลเดียวรองรับหลายระบบธุรกิจได้
ระบบจัดการฐานข้อมูล DBMS (Database Management System)	ระบบที่ช่วยผู้ใช้จัดการข้อมูลในฐานข้อมูล เช่น add, insert, update, delete; ตัวอย่าง Access, Oracle, MySQL

ข้อดีของสภาพแวดล้อมฐานข้อมูล (6 ข้อ)

Scalability ขยายได้	ระบบขยาย ปรับแก้ หรือลดขนาดได้
Economy of scale ประหยัดต่อขนาด	ออกแบบฐานข้อมูลทำให้ใช้ฮาร์ดแวร์ได้เต็มประสิทธิภาพขึ้น
Enterprise-wide application ใช้ทั้งองค์กร	DBA (Database administrator) ประเมินความต้องการโดยรวมและดูแลฐานข้อมูล
Stronger standards มาตรฐานเข้มแข็ง	ชื่อข้อมูล รูปแบบ และเอกสาร เป็นมาตรฐานเดียวกันทั้งองค์กร
Better security ความปลอดภัยดีขึ้น	เฉพาะผู้ใช้ที่ได้รับอนุญาตเข้าถึง และแต่ละคนมีระดับสิทธิ์ต่างกัน
Data independence อิสระของข้อมูล	ระบบที่ใช้ DBMS ไม่ต้องผูกกับวิธีเก็บข้อมูลจริง (physical) เปลี่ยนที่เก็บแล้วระบบไม่ต้องแก้

องค์ประกอบของ DBMS

Interfaces (3 กลุ่ม)	**Users** ใช้ predefined query และคำสั่ง switchboard หรือ query language; **Database administrators** รับผิดชอบการจัดการและสนับสนุน DBMS; **Related information systems** DBMS ให้บริการระบบอื่นที่เกี่ยวข้อง
Data Manipulation Language DML	ภาษาที่ควบคุมการทำงานกับฐานข้อมูล (เพิ่ม/แก้/ลบ/ค้น)
Schema	คำอธิบายของ **ทุก** field ตาราง และความสัมพันธ์ในฐานข้อมูล
Subschema	ส่วนของฐานข้อมูลที่ระบบหรือผู้ใช้คนหนึ่งต้องใช้/ได้รับอนุญาตให้เข้าถึง (มุมมองย่อยของ schema)
Physical data repository	ที่เก็บจริงซึ่งบรรจุ schema และ subschema; อยู่ที่เดียวหรือกระจายหลายที่ได้ ใช้ซอฟต์แวร์ที่รองรับ ODBC (open database connectivity) ให้ระบบและ DBMS ต่าง ๆ สื่อสารกัน

ฐานข้อมูลบนเว็บ: ฐานข้อมูลสร้างและจัดการด้วยภาษา/คำสั่งที่ไม่เกี่ยวกับ HTML จึงต้องเชื่อมฐานข้อมูลกับเว็บให้ดู/แก้ไขข้อมูลได้ โดยใช้ **middleware** (ตัวกลางเชื่อมแอปพลิเคชันต่างชนิดให้แลกเปลี่ยนข้อมูล); ข้อมูลต้องปลอดภัยแต่ผู้มีสิทธิ์เข้าถึงง่าย.

2.2 ศัพท์ข้อมูลและ Key entity, field, record, keys, referential integrity

ศัพท์	ความหมาย
Entity เอนทิตี	คน สถานที่ สิ่งของ หรือเหตุการณ์ที่เก็บข้อมูล เช่น CUSTOMER, ORDER
Table / File ตาราง	ชุดของ record ที่เก็บข้อมูลของ entity หนึ่ง
Field / Attribute ฟิลด์	ลักษณะหรือข้อเท็จจริงหนึ่งอย่างของ entity เช่น CustomerName
Tuple / Record เรคอร์ด (แถว)	ชุดของ field ที่อธิบาย 1 instance ของ entity (ลูกค้า 1 คน = 1 แถว)
Common field	attribute ที่ปรากฏในมากกว่า 1 entity ใช้เชื่อมตาราง
Repeating group	ชุด field ที่เกิดซ้ำกี่ครั้งก็ได้ในเรคอร์ดเดียว ค่าต่างกันในแต่ละครั้ง เช่น หลายสินค้าในใบสั่งซื้อเดียว
Functional dependence	field A ขึ้นกับ field B ถ้าค่าของ A ถูกกำหนดโดยค่าของ B (B → A) เช่น StudentID → StudentName
Associative entity	entity ที่เกิดจาก relationship M:N มี attribute ของตัวเอง เช่น ORDER LINE, GRADE

ลำดับ: field < record < table < database.

เปรียบเทียบ Key

Key	ความหมาย	ตัวอย่าง STUDENT(StudentID, NationalID, Name, AdvisorID)
Primary key PK	field หรือชุด field ที่ระบุ record ได้ **ไม่ซ้ำ และน้อยที่สุด** (minimal); 1 ตารางมี 1 PK; ขีดเส้นใต้	StudentID
Candidate key	field ใดก็ได้ที่ *มีสิทธิ์* เป็น PK (ไม่ซ้ำ); เลือก 1 ตัวเป็น PK ที่เหลือเป็น candidate	StudentID, NationalID (เลือก StudentID เป็น PK)
Combination key (composite)	PK ที่ประกอบด้วยหลาย field รวมกัน (ใช้เมื่อ field เดียวไม่ unique)	ORDER LINE(OrderNo, ProductNo, Qty)
Foreign key FK	field ในตารางหนึ่งที่ต้องตรงกับค่า PK ของอีกตาราง เพื่อสร้างความสัมพันธ์	AdvisorID ใน STUDENT ชี้ไป PK ของ ADVISOR
Secondary key	field/ชุด field ที่ใช้ *ค้นหา/เรียกดู* record (ไม่จำเป็นต้องไม่ซ้ำ)	Name หรือ Major ใช้ค้นนักศึกษา

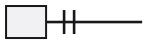
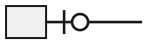
Referential integrity ความสมบูรณ์เชิงอ้างอิง	ชุดกฎที่ป้องกันข้อมูลไม่สอดคล้องกัน: ค่า FK ต้องมี PK ตรงกันในตารางที่อ้างถึง. ตัวอย่าง: เพิ่ม ORDER ของ CUSTOMER ที่ยังไม่มีในระบบไม่ได้; ลบ CUSTOMER ที่ยังมี ORDER ไม่ได้ (ต้องจัดการ ORDER ก่อน). Access มีตัวเลือก "Enforce referential integrity"

2.3 ERD: Entity-Relationship Diagram relationships, cardinality, crow's foot

วาด ERD: 1) ลิสต์ entity จากขั้นวิเคราะห์ (ดูจากคำนามในโจทย์) 2) พิจารณา relationship ระหว่างกัน 3) ใส่ cardinality 4) ถ้ามี M:N ให้แทรก associative entity. ชื่อ entity = **คำนามเอกพจน์**, ชื่อ relationship = **กริยา** อ่านเป็นประโยคอังกฤษได้ เช่น CUSTOMER places ORDER.

ชนิด	ความหมาย	ตัวอย่าง
1:1 one-to-one	แต่ละ instance ของ entity แรก มีของ entity ที่สองได้ **1 เดียว**	EMPLOYEE – SPOUSE (หนึ่งคู่สมรส)
1:M one-to-many	1 instance ของ entity แรก เกี่ยวกับ **หลาย** instance ของที่สอง แต่ที่สองแต่ละตัวเกี่ยวกับที่แรกได้ 1 เดียว	CUSTOMER – ORDER (ลูกค้า 1 คนหลายใบสั่งซื้อ)
M:N many-to-many	1 instance ของแต่ละฝั่งเกี่ยวกับ **หลาย** ของอีกฝั่ง → ต้องใช้ **associative entity** คั่น แตกเป็น 2 ความสัมพันธ์ 1:M	ORDER – PRODUCT ผ่าน ORDER LINE; STUDENT – COURSE ผ่าน GRADE

Cardinality (crow's foot) อ่านสัญลักษณ์ที่ปลายเส้นข้าง entity นั้น

สัญลักษณ์	ความหมาย	UML	จำง่าย
	One and only one	`1`	ขีดคู่ = หนึ่งเดียว
	One or many	`1..*`	ตีนกา + ขีด = อย่างน้อย 1
	Zero, one, or many	`0..*`	ตีนกา + วงกลม = 0 ถึงหลาย
	Zero or one	`0..1`	ขีด + วงกลม = 0 หรือ 1

วงกลม = ศูนย์ (ไม่บังคับ), ขีด = หนึ่ง, ตีนกา = หลาย. ตัวอย่างสไลด์: CUSTOMER ‖–o< ORDER = ลูกค้าหนึ่งเดียวสั่งได้ 0 ถึงหลายใบ; ORDER ‖–|< ITEM ORDERED = หนึ่งใบสั่งมี 1 ถึงหลายรายการ; EMPLOYEE ‖–o| SPOUSE = พนักงานมีคู่สมรส 1 คนหรือไม่มี; EMPLOYEE o<–o< PROJECT = หลายต่อหลาย ทั้ง 2 ฝั่งเป็นศูนย์ได้.

2.4 Normalization: UNF → 1NF → 2NF → 3NF normal forms

ใช้เมื่อ โจทย์ให้ตารางเดียวที่ข้อมูลซ้ำ/มี repeating group แล้วให้แยกเป็นตาราง หรือถาม normal form. เป้าหมาย: ลดความซ้ำซ้อน (ลดที่เก็บ), ลดข้อมูลไม่ตรงกัน, ลด anomaly การเพิ่ม/ลบ/แก้ไข, เพิ่มความคงทนของโครงสร้าง.

Form	เงื่อนไขที่ต้องผ่าน	ถ้าไม่ผ่านแก้ยังไง	ถามตัวเอง
UNF unnormalized	ยังมี repeating group	ไป 1NF	มีชุดข้อมูลซ้ำในเรคอร์ดเดียวไหม
1NF	ไม่มี repeating group	แตก 1 repeat = 1 record; **ขยาย PK** = PK เดิม + PK ของ repeating group	ทุกช่องมีค่าเดียว?
2NF	เป็น 1NF และ **ทุก field ที่ไม่ใช่ key ขึ้นกับ key ทั้งก้อน** (ไม่มี partial dependency)	แยก field ที่ขึ้นกับ *บางส่วนของ key* ออกเป็นตารางใหม่ ใช้ส่วนของ key นั้นเป็น PK	field นี้ขึ้นกับ key ครบหรือแค่ตัวใดตัวหนึ่ง
3NF	เป็น 2NF และ **ไม่มี non-key ขึ้นกับ non-key อื่น** (ไม่มี transitive dependency)	แยก field ที่ขึ้นกับ non-key ออกเป็นตารางใหม่ มี non-key ตัวนั้นเป็น PK (และคงไว้ในตารางเดิมเป็น FK)	field นี้ขึ้นกับ field ธรรมดาอื่นหรือไม่

จำ 3NF: ทุก field ขึ้นกับ **the key, the whole key, and nothing but the key**. ตาราง 1NF ที่ PK เป็น field เดียว = 2NF อัตโนมัติ (ไม่มี "บางส่วน" ของ key). ถ้า PK ไม่ใช่ combination key ข้ามขั้น 2NF ได้.

สัญกรณ์มาตรฐาน และขั้นตอนบนกระดาษคำตอบ

เขียน `NAME(PK, field2, field3)` ขีดเส้นใต้ PK; FK ทำเครื่องหมาย (FK). ขั้นตอน: (1) เขียน UNF ใส่ repeating group ใน { } (2) แปลง 1NF ขยาย PK (3) เขียน functional dependency ของทุก field (4) แยก 2NF (5) หา transitive แยก 3NF (6) ระบุ PK/FK (7) วาด ERD พร้อม cardinality (8) ตรวจ: ทุกตารางมี PK, เชื่อมด้วย FK, ไม่เก็บค่าที่คำนวณได้.

ตัวอย่างสไลด์ (ตัวอย่างสั้น)

ตัวอย่าง	ตาราง / ปัญหา	ผลการแยก
2NF	ABC(ชิ้นส่วน, ชื่อโกดัง, จำนวนสินค้า, ที่อยู่โกดัง)	ที่อยู่โกดังขึ้นกับ ชื่อโกดัง (บางส่วนของ key) → สินค้า(ชิ้นส่วน, ชื่อโกดัง, จำนวน) + โกดัง(ชื่อโกดัง, ที่อยู่โกดัง)
3NF	ผู้บริหาร(เลขประจำตัว, ชื่อนามสกุล, ที่อยู่, ตำแหน่ง, ยี่ห้อรถประจำตำแหน่ง)	ยี่ห้อรถขึ้นกับ ตำแหน่ง (non-key → non-key) → ผู้บริหาร(เลขประจำตัว, ชื่อนามสกุล, ที่อยู่, ตำแหน่ง) + ตำแหน่งบริหาร(ตำแหน่ง, ยี่ห้อรถประจำตำแหน่ง)
Crossroads College	STUDENT มี repeating group ของวิชาที่เรียน	1NF: PK = StudentNo + CourseNo. 2NF: STUDENT, COURSE, **GRADE** (entity ใหม่ที่พบตอน normalize เป็น associative entity). 3NF: ชื่อที่ปรึกษาขึ้นกับ AdvisorNo → แยก ADVISOR ออกจาก STUDENT
Order / Product	ORDER มี repeating group ของสินค้า	1NF: PK = Order + ProductNo. 2NF: ORDER, PRODUCT, **ORDER LINE**. 3NF: ข้อมูลผู้ขายขึ้นกับ Supplier (non-key) → แยก PRODUCT เป็น PRODUCT + SUPPLIER

ตัวอย่างทำเต็ม: คลินิกทันตกรรม "ยิ้มสดใส" (แบบฝึกหัด) ตารางเดียว 1 แถว = 1 นัดหมาย ในแถวมีรายการหัตถการหลายรายการ

Anomaly ของตารางเดิม: Insert ลงทะเบียนคนไข้/ทันตแพทย์/หัตถการใหม่ไม่ได้ถ้ายังไม่มีนัด; Update เปลี่ยนเบอร์คนไข้หรือค่าธรรมเนียมแผนกต้องแก้ทุกแถว พลาดแล้วข้อมูลขัดกัน; Delete ลบนัดสุดท้ายแล้วข้อมูลคนไข้/หัตถการหายไปด้วย; ข้อมูลซ้ำ (ชื่อคนไข้ ชื่อแผนกซ้ำทุกนัด) เปลืองที่.

```
UNF  APPOINTMENT(AppNo, AppDate, PatientID, PatientName, Address, Phone,
         DentistID, DentistName, DeptID, DeptName, DeptFee,
         {ProcID, ProcName, ProcCategory, StdTime, UnitPrice, Units})   ← repeating group

1NF  APPOINTMENT(AppNo, ProcID, AppDate, PatientID, PatientName, Address, Phone,
         DentistID, DentistName, DeptID, DeptName, DeptFee,
         ProcName, ProcCategory, StdTime, UnitPrice, Units)

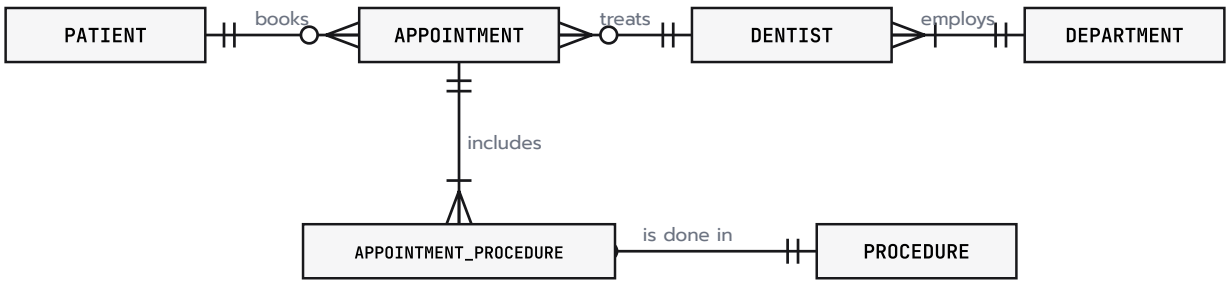
FD    AppNo → AppDate, PatientID, PatientName, Address, Phone, DentistID, DentistName,
                DeptID, DeptName, DeptFee                      (partial: ขึ้นกับ AppNo อย่างเดียว)
      ProcID → ProcName, ProcCategory, StdTime, UnitPrice   (partial: ขึ้นกับ ProcID อย่างเดียว)
      AppNo, ProcID → Units                                 (ขึ้นกับ key ทั้งก้อน)

2NF  APPOINTMENT(AppNo, AppDate, PatientID, PatientName, Address, Phone,
                  DentistID, DentistName, DeptID, DeptName, DeptFee)
      PROCEDURE(ProcID, ProcName, ProcCategory, StdTime, UnitPrice)
      APPOINTMENT_PROCEDURE(AppNo (FK), ProcID (FK), Units)

FD    PatientID → PatientName, Address, Phone
      DentistID → DentistName, DeptID
      DeptID → DeptName, DeptFee                (non-key → non-key = transitive)

3NF  PATIENT(PatientID, PatientName, Address, Phone)
      DEPARTMENT(DeptID, DeptName, DeptFee)
      DENTIST(DentistID, DentistName, DeptID (FK))
      APPOINTMENT(AppNo, AppDate, PatientID (FK), DentistID (FK))
      PROCEDURE(ProcID, ProcName, ProcCategory, StdTime, UnitPrice)
      APPOINTMENT_PROCEDURE(AppNo (FK), ProcID (FK), Units)
```

ราคารวมของรายการ = Units × UnitPrice เป็นค่าที่คำนวณได้ **ไม่เก็บ** ในตาราง. ERD หลัง 3NF พร้อม cardinality:



2.5 Data warehouse, Data mart, Data mining, Data control storage, access and control

	Data warehouse	Data mart	Data mining
คืออะไร	ที่รวมข้อมูลบูรณาการจากหลายระบบ แม้ดูไม่เกี่ยวข้องกัน ไม่ว่าเก็บที่ไหนในองค์กร	คลังข้อมูลย่อย ออกแบบตอบความต้องการของ **แผนกใดแผนกหนึ่ง**	การค้นหา **แบบแผน/ความสัมพันธ์** ที่มีความหมายในข้อมูล (เช่น clickstream)
ผู้ใช้ทำอะไร	เลือก data dimension แล้วดึงข้อมูลโดยไม่ต้องรู้ว่าเก็บที่ไหน	ใช้เฉพาะข้อมูลของแผนกนั้น	ใช้สร้างโปรไฟล์ลูกค้าใหม่ เพิ่มหน้าที่ดูต่อ session, ลดจำนวนคลิกถึงปิดการขาย, เพิ่ม checkout ต่อการเข้าชม และกำไรเฉลี่ยต่อ checkout
คำสำคัญ	ทั้งองค์กร	เฉพาะแผนก	หาแบบแผน (pattern)

Data control (การป้องกันข้อมูล)	1) จำกัดการเข้าถึงไฟล์/ฐานข้อมูล: user ID + password, permissions, encryption 2) เก็บ **backup** ไว้ตามกำหนดเวลา 3) มี **recovery** procedure กู้ไฟล์/ฐานข้อมูล 4) เก็บ audit log files และ audit fields
แนวคิดออกแบบฐานข้อมูล (อาจารย์)	ลดความซ้ำซ้อนเป็นหลัก; เพิ่ม/แก้ข้อมูลง่ายไม่ผิดพลาด; แยกเป็นหลายตารางที่สัมพันธ์กัน; เก็บเท่าที่จำเป็น; **ไม่เก็บค่าที่มาจากการคำนวณ**; ทุกตารางมี PK; เชื่อมตารางด้วย FK. หาตารางจาก **คำนามในโจทย์**

ตัวอย่างโจทย์ ABC (รายงานยอดขายตามสาขา รัฐ ประเทศ ไตรมาส ปี): นามในโจทย์ → ตาราง Sales(Sale_Id, PR_Id FK, Loc_Id FK, Sales, Date), Product(PR_Id, PR_name, Cat_Id FK), Category(Cat_Id, Cat_name), Location(Loc_Id, Loc_Code, Loc_Name, State_Id FK, Country_Id FK), State(State_Id, State_Name), Country(Country_Id, Country_Name). Sales อยู่ตรงกลาง (ข้อเท็จจริงที่นับ) ส่วน Product/Location/เวลา เป็นมิติที่ใช้แบ่งรายงานสำหรับคลังข้อมูล.

เกร็งข้อสอบ

โจทย์ถามแบบนี้	ตอบ / เลือกแบบนี้
ตารางนี้ได้ normal form ที่เท่าไร	มี repeating group = UNF; ไม่มี แต่ field ขึ้นกับ key บางส่วน = 1NF; ขึ้นกับ non-key อื่น = 2NF; ผ่านหมด = 3NF
PK เป็น field เดียว และไม่มี repeating group	ผ่าน 2NF อัตโนมัติ (partial dependency เกิดได้เฉพาะ combination key)
field ขึ้นกับบางส่วนของ composite key	ผิด 2NF: แยกตารางใหม่ (ตัวอย่าง ที่อยู่โกดังขึ้นกับชื่อโกดัง)
field ขึ้นกับ field ที่ไม่ใช่ key	ผิด 3NF: แยกตารางใหม่ (ตำแหน่ง → ยี่ห้อรถ)
relationship M:N	ต้องมี associative entity (ORDER LINE, GRADE); PK ของมัน = PK ทั้งสองฝั่งรวมกัน (combination key)
ควรเก็บยอดรวม/อายุที่คำนวณได้ไหม	ไม่เก็บ: ค่าที่มาจากการคำนวณไม่เก็บ
ความต่างของ key	Primary = ระบุไม่ซ้ำ; Candidate = มีสิทธิ์เป็น PK; Foreign = อ้างถึง PK ตารางอื่น; Secondary = ใช้ค้นหา (ซ้ำได้)
Referential integrity ยกตัวอย่าง	เพิ่ม ORDER ให้ CUSTOMER ที่ไม่มีอยู่ไม่ได้; ลบ CUSTOMER ที่ยังมี ORDER ไม่ได้
อ่าน crow's foot	วงกลม = 0, ขีด = 1, ตีนกา = หลาย; อ่านจากสัญลักษณ์ข้าง entity ที่ถามถึง
ข้อ ER-Diagram	entity = คำนามเอกพจน์, relationship = กริยา, ใส่ cardinality ทุกเส้น, M:N ต้องแตกด้วย associative entity
Data warehouse / mart / mining	warehouse = ทั้งองค์กร; mart = แผนกเดียว; mining = หาแบบแผน
ข้อดีของ DBMS / ฐานข้อมูล	ลดซ้ำซ้อน, scalability, economy of scale, มาตรฐาน, ความปลอดภัย, data independence
การป้องกันข้อมูล	จำกัดสิทธิ์ (ID/password/permission/encryption), backup, recovery, audit log/audit fields

3 กลยุทธ์การพัฒนาและความคุ้มค่า Development Strategies, Cost-Benefit

3.1 อินเทอร์เน็ตกับการพัฒนาระบบ: SaaS, Web 2.0, Cloud impact of the Internet

ศัพท์	ความหมาย
Software as a Service SaaS	ซอฟต์แวร์ถูกฝังเป็น **บริการบนอินเทอร์เน็ต** แทนการขาย license ให้ลูกค้าติดตั้งเอง (วิธีเดิม: ผู้พัฒนาขายซอฟต์แวร์ตรง ลูกค้าได้ license); ช่วยลดค่าบำรุงรักษาระบบ
Web 2.0	เว็บยุคที่ 2: ผู้ใช้แชร์ข้อมูลและโต้ตอบกันแบบ dynamic; สนับสนุน wiki, blog, social network (เช่น Facebook)
Cloud computing	การประมวลผลตามความต้องการของผู้ใช้: ผู้ใช้ระบุความต้องการ ซอฟต์แวร์ร้องขอให้ระบบจัดสรรทรัพยากรและบริการให้ตรงความต้องการ; เพิ่ม/ลดทรัพยากรได้ตลอดเวลา โดยผู้ใช้ไม่ต้องรู้เบื้องหลัง

	Traditional development	Web-based development
เป้าหมาย	ทำงานกับฮาร์ดแวร์/OS ที่ลูกค้าใช้อยู่ เน้นเครือข่ายท้องถิ่น (LAN, WAN)	ให้บริการผ่านอินเทอร์เน็ต มอง Web เป็นเหมือนระบบปฏิบัติการ (เช่น .NET)
ทางเลือกพัฒนา	3 ทาง: In-house development, Off-the-shelf, Outsource	บริษัท **ซื้อบริการ** มากกว่าซื้อตัวระบบ (เช่น ระบบสั่งซื้อ CRM จัดการวัตถุดิบ)
ข้อดี/ลักษณะ	ความปลอดภัยซับซ้อนน้อยกว่า; เหมาะกับระบบที่ไม่จำเป็นต้อง online	ขยายข้อมูลและผู้ใช้ง่ายกว่า ทำงานบนฮาร์ดแวร์ต่างกันได้
ข้อจำกัด/ต้องมี	แทนระบบเก่ายาก (ใช้ได้เมื่อไม่ต้องแทนที่)	ต้องใช้เครื่องมือใหม่ (IBM WebSphere, Microsoft .NET); ต้องสร้าง **middle layer** เชื่อมระบบเดิม

3.2 ทางเลือกการพัฒนา: In-house, Off-the-shelf, Outsource make or buy, outsourcing

ศัพท์	ความหมาย
In-house development	บริษัทพัฒนาซอฟต์แวร์เอง
Off-the-shelf	ซื้อซอฟต์แวร์สำเร็จรูปมาใช้
Horizontal application ซอฟต์แวร์ใช้ได้หลายองค์กร	ใช้ได้ในองค์กรหลายแบบ เช่น ซอฟต์แวร์บัญชี, Word
Vertical application ซอฟต์แวร์เฉพาะธุรกิจ	สร้างสำหรับธุรกิจใดธุรกิจหนึ่งโดยเฉพาะ เช่น ระบบธนาคาร ระบบโรงพยาบาล
Make or Buy (Build or Buy)	การตัดสินใจว่าจะพัฒนาเองหรือซื้อสำเร็จรูป

เหตุผลที่ควรพัฒนาเอง (Build)	เหตุผลที่ควรซื้อสำเร็จรูป (Buy)
ตรงกับความต้องการทางธุรกิจ	ต้นทุนต่ำกว่า
ป้องกันการเปลี่ยนกระบวนการทำงานของบริษัท	ใช้เวลาพัฒนาน้อยกว่า
ทำงานร่วมกับระบบเดิมได้	ต้องการคนพัฒนาน้อยกว่า
สอดคล้องกับความต้องการด้านความปลอดภัย	การอัปเกรดในอนาคตได้รับบริการจากผู้ขายซอฟต์แวร์

Outsourcing

Outsourcing	จ้างบริษัทอื่นพัฒนาระบบให้ บริษัทเหล่านี้เรียก **Service provider**; รูปแบบ: พัฒนาระบบ IT, ทำ Research, Case studies
Application Service Provider ASP	บริษัทที่ **สร้างหรือให้เช่าซอฟต์แวร์**
Offshore outsourcing	ย้ายการพัฒนาระบบ IT ไปจ้างบริษัทในต่างประเทศ นิยมประเทศค่าแรงถูก เช่น อินเดีย; ต้องพิจารณาการควบคุมดูแล วัฒนธรรมที่ต่างกัน การสื่อสาร ความปลอดภัยของข้อมูล
ควรจ้าง outsource เมื่อ	ประหยัดต้นทุน และผู้รับจ้างมีความชำนาญสูงและน่าเชื่อถือ; ต้องพิจารณาเพิ่ม: การซื้อขาย license, การประกันต่าง ๆ, การบริการหลังการขาย

รูปแบบค่าใช้จ่าย Outsource (3 แบบ)	**จ่ายตาม**
Fixed fee model	ค่าธรรมเนียมคงที่ตามสัญญา
Subscription model	จำนวนผู้ใช้ (#users)
Usage model / Transaction model	ปริมาณการใช้ / จำนวนธุรกรรม (volume of transaction)

3.3 ประมาณต้นทุนและผลตอบแทน cost-benefit analysis

ส่วนสุดท้ายของขั้นตอนการออกแบบ SA ต้องวิเคราะห์เรื่องเงินและความคุ้มค่า.

	Tangible benefit	**Intangible benefit**
ความหมาย	ผลตอบแทนที่ **แสดงให้เห็นชัดเจน** วัดเป็นตัวเงินได้	ผลตอบแทนที่ **แสดงให้เห็นชัดเจนไม่ได้** วัดเป็นเงินยาก
ตัวอย่าง (ทั่วไป)	ลดต้นทุน ลดจำนวนพนักงาน ยอดขายเพิ่ม	ความพึงพอใจของลูกค้า ภาพลักษณ์องค์กร การตัดสินใจดีขึ้น

วิธีวิเคราะห์การเงิน: **Payback analysis** (ระยะเวลาคืนทุน), **Discounted payback**, **NPV** (มูลค่าปัจจุบันสุทธิ). อัตราคิดลด r = ต้นทุนของเงินทุน; **time value of money** = เงินวันนี้มีค่ากว่าเงินอนาคต ต้อง "ลด" เงินอนาคตเป็นมูลค่าปัจจุบัน.

3.4 คำนวณ: Payback period (ระยะเวลาคืนทุน) simple payback

ใช้เมื่อ ถามว่าใช้กี่ปีจึงได้เงินลงทุนคืน และโจทย์ **ไม่** บอกอัตราคิดลด.

$$\text{Payback} = n + \frac{R}{CF}$$

สัญลักษณ์	**ความหมาย**	**หาจากไหน**
`n`	จำนวนปีเต็มก่อนคืนทุน	ปีสุดท้ายที่เงินสะสมยังติดลบ (บวกกระแสเงินสดทีละปีจากเงินลงทุน)
`R`	เงินที่ยังไม่คืน ณ สิ้นปี n	|เงินสะสมของปี n| (ค่าติดลบที่เหลือ)
`CF`	กระแสเงินสดรับของปี n+1 (ปีที่คืนทุน)	จากตารางรายได้ของปีที่เงินสะสมกลับเป็นบวก

1. ตั้งตาราง ปีที่ 0 = เงินลงทุนติดลบ
2. ปีละบรรทัด: เงินคงเหลือสะสม = ของปีก่อน + รายได้ปีนั้น
3. หาปีที่เงินสะสมเปลี่ยนจากลบเป็นบวก; ก่อนหน้านั้น n ปี; ใส่สูตร

ตัวอย่างสไลด์ Ex1: ลงทุน 105,000 บาท รายได้ปีละ 35,000 / 45,000 / 50,000 / 50,000 / 45,000 / 35,000

ปีที่	ลงทุน	รายได้	เงินคงเหลือ
0	-105,000		-105,000
1		35,000	-70,000
2		45,000	-25,000
3		50,000	25,000
4		50,000	75,000
5		45,000	120,000
6		35,000	155,000

สิ้นปี 2 ยังขาด 25,000; ปี 3 รายได้ 50,000 → Payback = 2 + 25,000/50,000 = **2.5 ปี** (2 ปี 6 เดือน).

ข้อดี	ข้อเสีย
บอกความเสี่ยงและสภาพคล่องของโครงการ; คำนวณและเข้าใจง่าย	ไม่คำนึงถึง time value of money; ไม่คำนึงถึงกระแสเงินสดหลังระยะเวลาคืนทุน

3.5 คำนวณ: มูลค่าปัจจุบัน PV และ PV factor present value

ใช้เมื่อ ต้องเปลี่ยนเงินในอนาคตเป็นเงินวันนี้ (ฐานของ Discounted payback และ NPV).

$$PV = \frac{V_t}{(1+r)^t} = V_t \times PV\ factor \qquad PV\ factor = \frac{1}{(1+r)^t}$$

สัญลักษณ์	ความหมาย	หาจากไหน
PV	Present value มูลค่าปัจจุบัน	คำตอบ (บาท ณ วันนี้)
V_t	กระแสเงินสดในปีที่ t	จากตารางรายได้/ต้นทุนของโจทย์
r	อัตราคิดลด discount rate (ต้นทุนเงินทุน)	โจทย์ให้ เช่น 12% = 0.12; ใส่เป็นทศนิยมในสูตร
t	ปีที่ (ปีปัจจุบัน t = 0)	ปีที่ 0 ไม่ต้องลด (factor = 1); ปีที่ 1 = 1 ปีจากนี้
$PV\ factor$	ตัวคูณลดค่า	อ่านจากตารางภาคผนวก (แถว = ปี t, คอลัมน์ = r) หรือคำนวณ 1/(1+r)^t; ปี 0 = 1

ตัวอย่างสไลด์: ผลประโยชน์ปีละ 300,000 บาท เป็นเวลา 5 ปี รวมปีปัจจุบัน (t = 0, 1, 2, 3, 4), r = 0.1

t	V_t	PV factor	$PV = V_t \times factor$
0	300,000	$1/1.1^0 = 1.0000$	300,000.00
1	300,000	$1/1.1^1 = 0.9091$	272,727.27
2	300,000	$1/1.1^2 = 0.8264$	247,933.88
3	300,000	$1/1.1^3 = 0.7513$	225,394.44
4	300,000	$1/1.1^4 = 0.6830$	204,904.04
รวม			**1,250,959.63**

PV รวม = **1,250,959.63** บาท. (ใช้ factor 4 ตำแหน่งจากตาราง: 300,000×(1+.9091+.8264+.7513+.6830) = 1,250,940; ต่างเล็กน้อยจากการปัดเศษ.) ตัวอย่างที่ 2: เงิน 3,000 บาทปีที่ 3 ที่ r = 11.4% = $3{,}000/1.114^3$ = 3,000 × 0.7233 = **2,170.03**.

ระวังการนับปี: "5 ปี รวมปีปัจจุบัน" = t 0 ถึง 4 (5 งวด); "ปีที่ 1 ถึง 5" = t 1 ถึง 5.

3.6 คำนวณ: Discounted payback period discounted payback

ใช้เมื่อ โจทย์ให้อัตราดอกเบี้ย/อัตราคิดลดและถามระยะเวลาคืนทุน. วิธี = ลดกระแสเงินสดเป็นมูลค่าปัจจุบันก่อน แล้วหา payback.

$$\text{Discounted payback} = n + \frac{R_{PV}}{PV_{n+1}}$$

สัญลักษณ์	ความหมาย	หาจากไหน
n	จำนวนปีเต็มก่อนคืนทุน	ปีสุดท้ายที่เงินสะสม **ของ PV** ยังติดลบ
R_{PV}	เงินที่ยังไม่คืน (หน่วย PV) ณ สิ้นปี n	|เงินสะสมปี n| จากคอลัมน์เงินคงเหลือ
PV_{n+1}	PV ของรายได้ปี n+1	รายได้ปีนั้น × PV factor ของปีนั้น

ตัวอย่างสไลด์ Ex2 (ข้อมูล Ex1, r = 12%; PV factor จากตาราง .8929 .7972 .7118 .6355 .5674 .5066)

ปีที่	ลงทุน	รายได้	PV 12%	ค่าเงินปัจจุบัน	เงินคงเหลือ
0	-105,000		1.0000	-105,000.0	-105,000.0
1		35,000	0.8929	31,251.5	-73,748.5
2		45,000	0.7972	35,874.0	-37,874.5
3		50,000	0.7118	35,590.0	-2,284.5
4		50,000	0.6355	31,775.0	29,490.5
5		45,000	0.5674	25,533.0	55,023.5
6		35,000	0.5066	17,731.0	72,754.5

สิ้นปี 3 ยังขาด 2,284.5; ปี 4 มี PV 31,775.0 → Discounted payback = 3 + 2,284.5/31,775.0 = **3.07 ปี**. เทียบ simple payback 2.5 ปี: discounted ยาวกว่าเสมอ (ต้องรอเงินที่ถูกลดค่า).

ตัวอย่างสไลด์ Project L (r = 10%): สไลด์พิมพ์ PV ปี 1 = 9.901 ซึ่งผิด (นั่นคือ factor ที่ 1%); ถูก = 10/1.1 = 9.091

ปีที่	กระแสเงินสด	PV factor 10%	ค่าเงินปัจจุบัน	เงินสะสม
0	-100	1.0000	-100.000	-100.000
1	10	0.9091	9.091	-90.909
2	60	0.8264	49.587	-41.322
3	80	0.7513	60.105	18.783
4	80	0.6830	54.641	73.424

Discounted payback = 2 + 41.322/60.105 = **2.69 ปี** (สไลด์ได้ 2.674 จากเลข 9.901 ที่ผิด).

3.7 คำนวณ: NPV (Net Present Value) decision rules

ใช้เมื่อ ถามว่าควรลงทุนในระบบนี้หรือไม่ / เลือกโครงการไหน, หรือถามมูลค่าสุทธิเป็นเงิน.

$$\text{NPV} = \text{PV(รับ)} - \text{PV(จ่าย)} = \sum_{t=0}^{T} \frac{(B_t - C_t)}{(1+r)^t}$$

สัญลักษณ์	ความหมาย	หาจากไหน
B_t	ประโยชน์ (กระแสเงินสดรับ) ปีที่ t	ตารางรายได้ของโจทย์
C_t	ต้นทุน (กระแสเงินสดจ่าย) ปีที่ t	เงินลงทุนปี 0 และต้นทุนปีอื่น (ถ้ามี)
r	อัตราคิดลด	โจทย์ให้
t, T	t = 0 คือปีปัจจุบัน, T = ปีสุดท้ายของโครงการ	นับจากตารางโจทย์

1. คิด PV ของรายรับทุกปี (รายได้ × PV factor)
2. คิด PV ของรายจ่ายทุกปี (เงินลงทุนปี 0 ไม่ต้องลด)

3. NPV = รวม PV รับ - รวม PV จ่าย
4. ตัดสินใจตามกฎด้านล่าง เขียนเหตุผลเป็นประโยค

ผล	ความหมาย	ตัดสินใจ
NPV > 0	ลงทุนแล้วมีกำไร มูลค่ากิจการเพิ่มขึ้น	**ลงทุน**
NPV = 0	ผลตอบแทนเท่ากับต้นทุนเงินทุนพอดี	เกณฑ์ NPV ≥ 0 ยอมรับได้ (ไม่ขาดทุนเชิงมูลค่า)
NPV < 0	ผลตอบแทนต่ำกว่าต้นทุนเงินทุน	**ไม่ลงทุน**
โครงการ **ทดแทนกันได้** (mutually exclusive)	เลือกได้อย่างเดียว	เลือกตัวที่ **NPV สูงกว่า**
โครงการ **อิสระต่อกัน** (independent)	ลงทุนได้ทุกโครงการ	ลงทุนทุกโครงการที่ **NPV เป็นบวก** (เช่น ทั้งโครงการ S และ L)

ตัวอย่าง A (สไลด์/ซีต 3): ลงทุน 1,000,000; รายได้ 300,000 ต่อปี t = 0 ถึง 4; r = 10%

t	รายได้	PV factor 10%	PV
0	300,000	1.0000	300,000
1	300,000	0.9091	272,730
2	300,000	0.8264	247,920
3	300,000	0.7513	225,390
4	300,000	0.6830	204,900
		PV รวม	**1,250,940**
		NPV = PV รวม - 1,000,000	250,940

ใช้สูตรตรง: PV รวม 1,250,959.63 → NPV = **250,959.63** > 0 (ซีตอาจารย์ใช้ factor 4 ตำแหน่ง = 250,940) → **ควรลงทุน**.

ตัวอย่าง B (ซีต 2): ลงทุน 12,900 บาท รายได้ 3,000 บาทต่อปี t = 0 ถึง 4; r = 12%

ปีที่	ลงทุน	รายได้	PV 12%	ค่าเงินปัจจุบัน	เงินคงเหลือ
0	-12,900	3,000	1.0000	3,000.0	-9,900.0
1		3,000	0.8929	2,678.7	-7,221.3
2		3,000	0.7972	2,391.6	-4,829.7
3		3,000	0.7118	2,135.4	-2,694.3
4		3,000	0.6355	1,906.5	-787.8

PV รายได้รวม = 12,112.2; NPV = 12,112.2 - 12,900 = **-787.8** < 0 (สิ้นปีสุดท้ายยังคืนทุนไม่หมด เหลือ 787.80) → **ไม่ควรลงทุน**.

เปรียบเทียบ 3 วิธี

	Payback	Discounted payback	NPV
time value of money	ไม่คิด	คิด	คิด
กระแสเงินสดหลังคืนทุน	ไม่คิด	ไม่คิด	คิดทุกปี
ผลลัพธ์	จำนวนปี	จำนวนปี (≥ payback ธรรมดา)	จำนวนเงิน (บาท)
ใช้อัตราคิดลด r	ไม่ใช้	ใช้	ใช้
เกณฑ์ตัดสิน	คืนทุนเร็ว/ภายในที่กำหนด	คืนทุนภายในที่กำหนด	NPV > 0 ลงทุน
จุดเด่น	ง่าย วัดความเสี่ยง/สภาพคล่อง	ง่าย + ลดค่าเงินแล้ว	วัดมูลค่าที่เพิ่ม ใช้เลือกโครงการ

เกร็งข้อสอบ

โจทย์ถามแบบนี้	ตอบ / เลือกแบบนี้
ใช้วิธีไหน	ไม่มี r = Payback; มี r ถามกี่ปี = Discounted payback; มี r ถามควรลงทุนไหม/เท่าไร = NPV
ปี 0 ลดค่าไหม	ไม่ (factor = 1); เงินลงทุนปี 0 ติดลบเต็มจำนวน
"5 ปี รวมปีปัจจุบัน"	t = 0,1,2,3,4 (5 งวด) ไม่ใช่ 1 ถึง 5
payback ข้อเสีย	ไม่คิดมูลค่าเงินตามเวลา และไม่คิดกระแสเงินสดหลังคืนทุน; ข้อดี: ง่าย บอกความเสี่ยง/สภาพคล่อง
Discounted กับ Simple payback	discounted มากกว่า (นานกว่า) เสมอ เช่น 3.07 vs 2.5 ปี
ตัดสินใจ NPV	NPV > 0 ลงทุน, < 0 ไม่ลงทุน, = 0 ผลตอบแทนเท่าต้นทุนเงินทุนพอดี; ทดแทนกัน = เลือก NPV สูงสุด; อิสระ = ลงทุนทุกโครงการที่ NPV บวก
ใช้ตัวคูณจากตาราง	อ่านที่ แถว = ปี, คอลัมน์ = อัตรา; เลข 4 ตำแหน่งอาจต่างจากสูตรเล็กน้อย (ยอมรับได้); เขียนที่มาให้ครบ
ผลตอบแทนแบบ tangible / intangible	tangible = วัดเป็นเงินได้ชัดเจน; intangible = วัดชัดเจนไม่ได้ (ความพึงพอใจ ภาพลักษณ์)
horizontal / vertical	horizontal = ใช้ได้หลายองค์กร (บัญชี, Word); vertical = เฉพาะธุรกิจ (ธนาคาร โรงพยาบาล)
จ่ายค่า outsource แบบไหน	คงที่ = fixed fee; ตามจำนวนผู้ใช้ = subscription; ตามปริมาณธุรกรรม = usage/transaction
ควรซื้อสำเร็จรูปเมื่อ	ต้นทุนต่ำ, เร็ว, ใช้คนน้อย, ผู้ขายอัปเกรดให้; **พัฒนาเอง**เมื่อ ตรงธุรกิจ, ไม่ต้องเปลี่ยนกระบวนการ, ทำงานร่วมระบบเดิม, ตรงความปลอดภัย
SaaS / Cloud / ASP	SaaS = ซอฟต์แวร์เป็นบริการบนเน็ต; Cloud = ทรัพยากรตามต้องการ; ASP = ผู้สร้าง/ให้เช่าซอฟต์แวร์; Offshore = จ้างต่างประเทศ (ดูแล วัฒนธรรม สื่อสาร ความปลอดภัย)
ข้อผิดในสไลด์ Project L	ปี 1 ที่ 10% = 9.091 (สไลด์พิมพ์ 9.901); payback ถูก = 2.69 ปี

4 Output, User Interface และ Input Output, UI, Input Design

4.1 การออกแบบ Output output design

Output และ UI เป็นขั้นแรกของขั้นออกแบบ. Output คำนึงถึงความต้องการผู้ใช้เป็นหลัก (หน้าจอ/รายงาน); UI เน้นการโต้ตอบระหว่างระบบกับผู้ใช้ (ขั้นตอนการใช้งาน). Output พื้นฐานคือ **หน้าจอและเครื่องพิมพ์**; output ของระบบหนึ่งเป็น input ของอีกระบบได้.

ถามตัวเองก่อนออกแบบ output 5 ข้อ: 1) จุดประสงค์คืออะไร 2) ใครใช้ สำคัญอย่างไร ใช้อย่างไร 3) มีข้อมูลอะไรบ้าง 4) แสดงทางช่องทางใด (จอ/เครื่องพิมพ์) 5) เป็นความลับหรือไม่.

ประเภท output	รายการ
Internet-based information delivery	Webcast, E-mail, Blogs, Instant Messaging, Wireless devices
Digital audio, images, video	Podcasts
Automated facsimile	Faxback systems
Computer output microfilm / digital media	COM; computer output to digital media
Specialized forms of output	Retail point-of-sale (POS), ATM, Plotter

4.2 รายงาน: ประเภท ส่วนประกอบ การควบคุม report types and design

ชนิด	แสดงอะไร	ใครใช้ / เมื่อไหร่ใช้
Detail report	ข้อมูลหลาย ๆ บรรทัด ทุกรายการ (1 รายการ = 1 บรรทัดหรือมากกว่า)	ผู้ปฏิบัติงานที่ต้องดูรายละเอียดทุกรายการ
Exception report	เฉพาะข้อมูลที่ **ผิดปกติ หรือตรงเงื่อนไขที่กำหนด** (เช่น สต็อกต่ำกว่าจุดสั่งซื้อ, ค้างชำระเกินกำหนด)	ผู้จัดการที่ต้องจัดการกับสิ่งผิดปกติ ไม่อยากอ่านทั้งหมด
Summary report	รายงานสรุป (ยอดรวม ไม่ลงรายการ)	เหมาะกับ **ผู้บริหารระดับสูง**

ส่วนประกอบรายงาน report layout	Report header ชื่อรายงาน/วันที่; Page header หัวคอลัมน์ของทุกหน้า; Detail lines บรรทัดข้อมูล; **Group footer** ผลรวมย่อย (subtotals) เมื่อเปลี่ยนกลุ่ม; Report footer ผลรวมทั้งหมด (grand totals); Page footer เลขหน้า
Control field / Control break	ฟิลด์ที่ใช้จัดกลุ่ม (เช่น STORE NUMBER); เมื่อค่าของ control field เปลี่ยน (control break) ระบบพิมพ์ subtotal ของกลุ่มนั้น
การมีส่วนร่วมของผู้ใช้	ให้ผู้ใช้ให้คำแนะนำก่อนใช้จริง: **Mock-up / Prototype** ตัวอย่างรายงานที่จะได้จากระบบ; **Report analysis form** ตัวอย่างรายงานพร้อมรายละเอียดของแต่ละส่วน
หลักการออกแบบรายงาน	อ่านง่ายและโครงสร้างดี; **Field order** เรียงลำดับข้อมูลในฟิลด์; **Consistent design** รูปแบบสม่ำเสมอ; เครื่องมือออกแบบรายงานมีมาให้ในเครื่องมือจัดการฐานข้อมูล
Output control and security	output ต้อง **ถูกต้อง สมบูรณ์ เป็นปัจจุบัน และเป็นความลับ** (เช่น จำกัดผู้รับ ทำลายรายงานที่ไม่ใช้ด้วย shredder)

4.3 คำนวณ: จำนวนหน้ารายงานและเวลาพิมพ์ report pages and print time

ใช้เมื่อ โจทย์ให้จำนวนรายการ จำนวนบรรทัดต่อหน้า header/footer ผลรวมกลุ่ม และความเร็วเครื่องพิมพ์ แล้วถามจำนวนแผ่นกับเวลา.

$$N = D + G \times s + T \qquad U = L - h - f$$

$$P = \left\lceil \frac{N}{U} \right\rceil \text{ (ปัดขึ้นเสมอ)} \qquad \text{เวลา} = \frac{P}{ppm} \text{ นาที}$$

สัญลักษณ์	ความหมาย	หาจากไหน
D	จำนวนบรรทัดรายการ (detail lines)	1 รายการ 1 บรรทัด = จำนวนรายการ
G	จำนวนกลุ่ม	เช่น 5 หมวดสินค้า
s	บรรทัดผลรวมต่อกลุ่ม	ที่โจทย์บอก (เช่น 3)
T	บรรทัดผลรวมทั้งหมด (grand total)	ที่โจทย์บอก (เช่น 3)
N	บรรทัดทั้งหมดที่ต้องพิมพ์	D + G×s + T
L, h, f	บรรทัดต่อหน้า / หัวกระดาษ / ท้ายกระดาษ	โจทย์ให้ (80, 2, 2)
U	บรรทัดใช้พิมพ์ได้จริงต่อหน้า	L - h - f
P	จำนวนหน้า = จำนวนแผ่น (ถ้า 1 หน้าต่อ 1 แผ่น)	ปัดขึ้นเป็นจำนวนเต็ม (หน้าสุดท้ายที่ไม่เต็มก็ใช้กระดาษ 1 แผ่น)
ppm	pages per minute ความเร็วเครื่องพิมพ์	โจทย์ให้ (16 PPM); เวลา (นาที) × 60 = วินาที

ตัวอย่างจากชีต: ขายเดือนกันยายน 1,880 รายการ (1 รายการ 1 บรรทัด); กระดาษ B4 พิมพ์ได้ 80 บรรทัดต่อหน้า ใช้เป็น header 2 + footer 2; ผลรวมแต่ละหมวด 3 บรรทัด มี 5 หมวด; ยอดรวมทั้งหมด 3 บรรทัด; เครื่องเลเซอร์ 16 PPM; 1 หน้าต่อ 1 แผ่น.

ขั้น	คำนวณ	ผล
บรรทัดใช้ได้ต่อหน้า U	80 - 2 - 2	76
บรรทัดผลรวมหมวด	5 × 3	15
บรรทัดทั้งหมด N	1,880 + 15 + 3	1,898
จำนวนหน้า	1,898 / 76 = 24.97 → ปัดขึ้น	25 หน้า
จำนวนแผ่น (1 หน้า/แผ่น)	= จำนวนหน้า	25 แผ่น
เวลา	25 / 16 = 1.5625 นาที; 0.5625 × 60 = 33.75 วินาที	1 นาที 33.75 วินาที (≈ 34 วินาที)

ตอบ: ใช้กระดาษ **25 แผ่น**; ใช้เวลา **1 นาที 33.75 วินาที** (1.5625 นาที).

4.4 หน้าจอผู้ใช้: HCI, User rights, Guidelines, Controls user interface design

ศัพท์	ความหมาย
Human-Computer Interaction HCI	อธิบายความสัมพันธ์ระหว่างคอมพิวเตอร์และผู้ใช้
Graphical user interface GUI	ส่วนติดต่อผู้ใช้แบบกราฟิก; เป้าหมายหลัก = หน้าจอที่ **ใช้ง่าย เรียนรู้ได้เร็ว**
User-centered design	ออกแบบโดยยึดผู้ใช้เป็นศูนย์กลาง: ใช้หลักออกแบบ 8 ข้อ, เข้าใจฟังก์ชันการทำงานขององค์กร, คิดว่าตัวเองเป็นผู้ใช้
Process-control (หน้าจอ)	หน้าจอทำหน้าที่ควบคุมการทำงานของผู้ใช้ (ขั้นตอนงาน)

User rights 10 ข้อ (IBM)

สิทธิ์	ผู้ใช้มีสิทธิ์ได้อะไร
1 Perspective	ผู้ใช้ถูกเสมอ ถ้าใช้ระบบแล้วมีปัญหา ระบบผิด ไม่ใช่ผู้ใช้
2 Installation	ติดตั้ง/ถอนฮาร์ดแวร์และซอฟต์แวร์ได้ง่าย ไม่มีผลเสียตามมา
3 Compliance	ระบบทำงานได้ตรงตามที่สัญญา
4 Instruction	มีคำแนะนำใช้ง่าย (user guide, online/contextual help, error message) ช่วยให้ถึงเป้าหมายและกู้จากปัญหาได้
5 Control	ควบคุมระบบได้ และทำให้ระบบตอบสนองคำขอ
6 Feedback	ระบบให้ข้อมูลชัดเจน เข้าใจง่าย ถูกต้อง เรื่องงานที่ทำและความคืบหน้า
7 Dependencies	ได้รับแจ้งชัดเจนว่าต้องมีอะไรบ้างจึงใช้ซอฟต์แวร์/ฮาร์ดแวร์สำเร็จ (system requirements)
8 Scope	รู้ขอบเขตและข้อจำกัดความสามารถของระบบ
9 Assistance	ติดต่อผู้ให้บริการเทคโนโลยีและได้รับการตอบสนองที่ดีเมื่อมีข้อกังวล
10 Usability	ผู้ใช้เป็นนายของเทคโนโลยี ไม่ใช่กลับกัน ผลิตภัณฑ์ใช้ง่ายเป็นธรรมชาติ

Guidelines การออกแบบหน้าจอ 8 ข้อ

1. ให้ความสำคัญกับวัตถุประสงค์ของระบบ
2. ออกแบบให้เข้าใจ เรียนรู้ และใช้งานง่าย
3. มีการทำงานที่เพิ่มประสิทธิภาพการทำงานของพนักงาน
4. ช่วยให้ผู้ใช้ตรวจสอบข้อผิดพลาดได้ง่าย
5. ให้ผู้ใช้ใส่ข้อมูลน้อยที่สุด
6. ให้ข้อมูลตอบกลับ (feedback) แก่ผู้ใช้
7. อนุญาตให้ผู้ใช้เลือกรูปแบบหน้าจอ เช่น layout ได้
8. ใช้ศัพท์พื้นฐาน ไม่ใช้ technical term

User interface controls 12 แบบ

Control	คืออะไร	Control	คืออะไร
Menu bar	แถบเมนูคำสั่งด้านบน	Drop-down list box	รายการที่กางลงมาให้เลือก 1 ค่า
Toolbar	แถบปุ่มทางลัด/ไอคอนคำสั่ง	Option button / radio button	เลือกได้ **1 ข้อ** จากกลุ่ม
Command button	ปุ่มสั่งงาน (OK, Find, Print)	Check box	เลือกได้ **หลายข้อ** (ติ๊กถูก)
Dialog box	หน้าต่างโต้ตอบขอข้อมูล/ยืนยัน	Calendar control	ปฏิทินให้เลือกวันที่
Text box	ช่องพิมพ์ข้อความ/ตัวเลข	Switchboard	หน้าเมนูหลักที่มีปุ่มไปยังแต่ละงาน
Toggle button	ปุ่มสลับสถานะ เปิด/ปิด	List box (scroll bar)	กล่องรายการเลื่อนดูได้ด้วย scroll bar

4.5 การออกแบบ Input input design, validation

ศัพท์	ความหมาย
GIGO Garbage in, garbage out	คุณภาพของ output ขึ้นกับข้อมูล input; ข้อมูลเข้าเสียก็ได้ผลลัพธ์เสีย
Data capture / Data entry	Data capture = การเก็บข้อมูล (รวบรวมจากต้นทาง); Data entry = กระบวนการนำข้อมูลเข้าระบบ
Source document	เอกสารต้นฉบับที่กรอกข้อมูล: ลำดับกรอกควรตามทิศการอ่าน ซ้ายไปขวา บนลงล่าง; บนเว็บท้าทายกว่าเพราะผู้ใช้อ่านต่างกัน

	Batch input	Online input
วิธี	รวมข้อมูลเป็นชุด แล้วป้อนตามกำหนดเวลา	ป้อนเข้าระบบทันทีขณะเกิดรายการ
ข้อมูลเป็นปัจจุบัน	ล่าช้ากว่า (ตามรอบ)	ทันที
ตรวจ error	พบทีหลัง ตอนประมวลผลชุด (ใช้ batch control)	ตรวจและแจ้งทันทีที่กรอก

ลดปริมาณ Input (Input volume) 4 ข้อ

1. ใส่เฉพาะข้อมูลที่จำเป็น
2. ไม่ใส่ input ที่ระบบคำนวณได้เอง
3. ไม่ใส่ค่าคงที่
4. ใส่ **รหัสแทนคำเต็ม**

ออกแบบหน้าจอรับข้อมูล (Data entry screen)

- ระบบบอกตำแหน่งที่ต้องกรอก; ทุกฟิลด์มี **caption** และบอกชนิดข้อมูลที่ใส่ได้
- แสดงรูปแบบการกรอก/สัญลักษณ์ช่วยตรวจ เช่น XX-XXX-XX
- กด Enter หรือ Tab เพื่อจบแต่ละฟิลด์; ค่าศูนย์/ทศนิยมศูนย์ ไม่ต้องให้ผู้ใช้ใส่
- แสดงค่าเริ่มต้น (default) หรือค่าที่พบมากสุด; แสดงค่าที่เป็นไปได้ในฟิลด์ที่มีค่าจำกัด (เช่น drop-down)
- ออกจากระบบได้ตลอดแม้ใส่ไม่ครบ; ให้ผู้ใช้ **ยืนยันความถูกต้อง** ก่อนบันทึก
- layout สอดคล้องกับเอกสารต้นฉบับ; ผู้ใช้ add, change, delete, view record ได้
- ใช้สีแยกชนิดฟิลด์: **ระบบสร้างให้** / **ผู้ใช้ป้อน** / **ระบบดึงหรือคำนวณ** (ตัวอย่างหน้าจอ CustOrders)

Data validation checks 8 แบบ

Check	ตรวจอะไร	ตัวอย่างที่ใช้ตอบ
1 **Sequence check**	ลำดับ/ความต่อเนื่องของข้อมูล	เลขที่ใบสั่งซื้อต้องเรียงต่อเนื่อง ไม่ขาด/ซ้ำ
2 **Existence check**	ฟิลด์ที่บังคับกรอก มีข้อมูลหรือไม่ (ไม่ว่าง)	ต้องกรอกชื่อลูกค้า
3 **Data type check**	ชนิดข้อมูลถูกต้อง	จำนวนเงินต้องเป็นตัวเลข ไม่มีตัวอักษร
4 **Range check** (limit check)	ความยาวหรือช่วงของค่าอยู่ในขอบเขตตายตัว	เดือน 1 ถึง 12; คะแนน 0 ถึง 100
5 **Reasonableness check**	ค่า *สมเหตุสมผล* ตามบริบท แม้ยังอยู่ในช่วง	ชั่วโมงทำงาน 24 ชั่วโมงต่อวันทุกวัน; ซื้อ 10,000 ชิ้นจากลูกค้ารายย่อย
6 **Validity check** (referential integrity)	ค่ามีอยู่จริงในตาราง/รายการที่อนุญาต	รหัสลูกค้าต้องมีใน CUSTOMER; รหัสสินค้าต้องอยู่ใน PRODUCT
7 **Combination check**	ใช้ **2 ฟิลด์ขึ้นไปร่วมกัน** ตรวจ	รหัสไปรษณีย์ต้องตรงกับจังหวัด; ประเภทสินค้ากับจำนวนที่ขายได้
8 **Batch controls** (hash totals)	จำนวนข้อมูลทั้งหมดของชุด/ผลรวมควบคุมต้องตรงกับยอดที่ตั้งไว้	นับใบสั่งซื้อในชุดได้ 50 ใบ ต้องตรงยอดควบคุม 50; hash total = ผลรวมรหัสที่ไม่มีความหมายทางบัญชี

Input control ทุกฟิลด์ต้องตรวจสอบได้: **Audit trail** บอกที่มาของข้อมูล (ใคร key เมื่อไหร่ จากกระบวนการไหน); **Data security** ป้องกันความเสียหายต่อข้อมูล; **Encryption** ข้อมูลบางอย่างต้องเข้ารหัส

เกร็งข้อสอบ

โจทย์ถามแบบนี้	ตอบ / เลือกแบบนี้
รายงานให้ผู้บริหารระดับสูง	Summary report; ผู้จัดการจัดการสิ่งผิดปกติ = Exception report; ต้องการทุกรายการ = Detail report
รายงานเฉพาะสินค้าสต็อกต่ำ/ตรงเงื่อนไข	Exception report
โจทย์พิมพ์รายงาน	U = L - header - footer; N = รายการ + (กลุ่ม×บรรทัดรวม) + grand total; หน้า = ปัดขึ้น(N/U); เวลา = หน้า/ppm
คำนวณหน้าแล้วได้ทศนิยม	ปัดขึ้นเสมอ (24.97 → 25)

โจทย์ถามแบบนี้	ตอบ / เลือกแบบนี้
เวลาพิมพ์เป็นนาทีทศนิยม	0.5625 นาที × 60 = 33.75 วินาที; ตอบ "1 นาที 33.75 วินาที"
เลือก validation check จากสถานการณ์	ลำดับต่อเนื่อง = sequence; ห้ามเว้นว่าง = existence; ต้องเป็นตัวเลข = data type; อยู่ในช่วง min-max = range; ดูเป็นไปไม่ได้จริง = reasonableness; ต้องมีในตารางอื่น = validity; 2 ฟิลด์เข้ากันไหม = combination; นับ/ยอดรวมชุด = batch control / hash total
Range vs Reasonableness	range = ขอบเขตตายตัวที่กำหนด; reasonableness = ค่าถูกช่วงแต่ผิดสามัญสำนึก
ลด input	ใส่เฉพาะจำเป็น, ไม่ใส่ค่าที่คำนวณได้, ไม่ใส่ค่าคงที่, ใช้รหัสแทนคำเต็ม
Batch vs Online	batch = รวมชุดป้อนตามเวลา; online = ป้อนทันที ตรวจ error ทันที
Audit trail คืออะไร	บอกที่มาของข้อมูล: ใคร key เมื่อไหร่ จากกระบวนการไหน
GIGO	input เสีย output ก็เสีย: จึงต้อง validation
User rights / UI guidelines	ท่อง 10 สิทธิ์ (Perspective, Installation, Compliance, Instruction, Control, Feedback, Dependencies, Scope, Assistance, Usability) และ 8 guidelines; ตอบตามสไลด์ครบทุกข้อ
เลือก control	เลือก 1 จากหลายข้อ = radio button; เลือกได้หลายข้อ = check box; ค่าจำกัด = drop-down; วันที่ = calendar; เปิด/ปิด = toggle; เมนูหลัก = switchboard
ก่อนออกแบบ output ถามอะไร	จุดประสงค์, ผู้ใช้/ความสำคัญ/การใช้, ข้อมูลอะไร, ช่องทางแสดง, เป็นความลับไหม

5 ส่งมอบระบบ System Delivery: Test, Train, Convert, Changeover, Maintain

5.1 เครื่องมือพัฒนาและการทดสอบระบบ 3 ระดับ application development, testing

เครื่องมือที่ใช้ตอนพัฒนาโปรแกรม (System development tools): **ERD, Flowcharts, Pseudocode, Decision tables และ Decision trees**.

ระดับ	ทดสอบอะไร	ลำดับ	ตัวอย่าง
Unit testing	แต่ละโปรแกรม/โมดูล **ทีละตัวโดดเดี่ยว** ว่าทำงานถูกต้อง	1	ทดสอบโปรแกรมคิดภาษีตัวเดียว ให้ข้อมูลเข้าและดูผลออก
Integration testing	โปรแกรม/โมดูล **ตั้งแต่ 2 ตัวขึ้นไปที่ทำงานร่วมกัน** ส่งข้อมูลถึงกันถูกต้อง	2	โปรแกรมรับใบสั่งซื้อส่งข้อมูลให้โปรแกรมตัดสต็อกได้ถูกต้อง
System testing	**ระบบทั้งหมด** รวมทุกโปรแกรม ส่วนเชื่อมต่อ ขั้นตอนงาน และผู้ใช้ ว่าทำตามความต้องการ	3	ทดสอบตั้งแต่รับออเดอร์ถึงออกใบแจ้งหนี้ด้วยข้อมูลจริง

ภาพสไลด์: วงกลมแต่ละวง = unit test ของแต่ละโปรแกรม; ลูกศรระหว่างวงที่เชื่อมกัน = integration test; กรอบรวมทั้งหมด = system test. ทดสอบจากเล็กไปใหญ่.

5.2 การติดตั้งและประเมินระบบ installation and evaluation

ขั้นตอนที่เหลือก่อนใช้จริง 5 อย่าง: 1) เตรียมสภาพแวดล้อมสำหรับทดสอบ 2) อบรมผู้ใช้ทั่วไป หัวหน้างาน และฝ่าย IT 3) แปลงข้อมูลให้ใช้กับระบบใหม่ได้ 4) ทดสอบระบบก่อนใช้งานจริง 5) ทำรายงานการติดตั้งและทดสอบให้หัวหน้า.

Operational and Test environments สภาพแวดล้อมใช้งานจริง / ทดสอบ	แยกสภาพแวดล้อมทดสอบจากของใช้งานจริง; เตรียม **ฮาร์ดแวร์ ซอฟต์แวร์** ให้พร้อมและ **ระบบสื่อสาร** ต่าง ๆ

5.3 การอบรม training

ศัพท์	ความหมาย
Training plan แผนอบรม	คิดก่อนว่าจะอบรมให้ใคร; ปกติมี **3 กลุ่ม**: **users** (ผู้ใช้), **managers** (ผู้จัดการ/หัวหน้า), **IT staff** (ฝ่าย IT)
Vendor training อบรมโดยผู้ขาย	อบรมโดยเจ้าของซอฟต์แวร์
Webinars, Podcasts, Tutorials	อบรมผ่านสื่อออนไลน์
Outside training resources แหล่งอบรมภายนอก	เช่าสถานที่ฝึกอบรม เช่น บริษัทหรือมหาวิทยาลัย
In-house training อบรมภายใน	ฝ่าย IT และแผนกผู้ใช้มักแบ่งความรับผิดชอบร่วมกัน
User training package	ชุดอบรมผู้ใช้; พนักงานใหม่ต้องอบรมระบบด้วย; อบรมเรื่องการเปลี่ยนแปลงระบบคล้ายอบรมครั้งแรก; เป้าหมาย = ให้เห็นว่าระบบช่วยทำงานได้อย่างไร

5.4 การแปลงข้อมูล data conversion

1. **Export** ข้อมูลจากระบบเก่าออกมาในรูปแบบที่ระบบใหม่นำเข้าได้
2. ถ้า export เป็นรูปแบบที่ระบบใหม่รู้จักไม่ได้ ต้อง **เขียนโปรแกรมแปลง** อ่านข้อมูลเก่าเข้าไปไว้ในระบบใหม่
3. ต้องมีวิธี **ตรวจสอบ** ว่าข้อมูลในระบบใหม่ถูกต้องหรือไม่

5.5 การเปลี่ยนระบบ (System changeover) 4 วิธี direct, parallel, pilot, phased

วิธี	ทำอย่างไร	ความเสี่ยง	ค่าใช้จ่าย	เงื่อนไข / ข้อจำกัด
Direct cutover (เปลี่ยนทันที)	หยุดระบบเก่าแล้วใช้ระบบใหม่ทันที	**สูงสุด**	**ต่ำสุด** (ประหยัดสุด)	ต้องแน่ใจว่าไม่มีข้อผิดพลาด; นิยมกับซอฟต์แวร์สำเร็จรูปทั่วไป เช่น MS Office
Parallel operation (ใช้คู่ขนาน)	รันระบบเก่าและใหม่ **พร้อมกัน** ช่วงหนึ่ง แล้วค่อยเลิกระบบเก่า	**ต่ำสุด** (น้อยกว่า direct)	**สูงสุด** (รัน 2 ระบบ สูงกว่า direct)	ใช้ไม่ได้ถ้าระบบใหม่ทำงานร่วมกับระบบเก่าไม่ได้
Pilot operation (ทดลองบางหน่วย)	ใช้ระบบใหม่เฉพาะบางส่วนของหน่วยงาน (**pilot site**) ก่อน; ไม่พบข้อผิดพลาด จึงเปลี่ยนที่อื่นต่อแบบ **direct cutover**	ปานกลาง	ปานกลาง	แบ่งตาม **สถานที่/หน่วยงาน**
Phased operation (ทีละโมดูล)	เปลี่ยนไปใช้ระบบใหม่ **ทีละ module**	ปานกลาง (น้อยกว่า parallel)	ปานกลาง (ประหยัดกว่า parallel)	ใช้ไม่ได้ถ้าระบบแบ่งเป็นโมดูลไม่ได้; แบ่งตาม **โมดูล/ฟังก์ชัน**

ความเสี่ยง (สูง → ต่ำ)	Direct cutover	Pilot / Phased	Parallel
ค่าใช้จ่าย (ต่ำ → สูง)	ต่ำ	กลาง	สูง
ความเสี่ยง	สูง	กลาง	ต่ำ

จำ: เสี่ยงมาก = ถูก (direct); ปลอดภัยมาก = แพง (parallel); pilot = ลองก่อน **ที่ไซต์เดียว** แล้ว direct ที่เหลือ; phased = ทีละ **โมดูล**.

5.6 หลังการติดตั้ง และการสนับสนุนผู้ใช้ post-implementation, user support

ศัพท์	ความหมาย
Final report to management รายงานสรุปถึงผู้บริหาร	**3 อย่าง**: 1) เอกสารระบบฉบับสมบูรณ์ทั้งหมด (user manual, admin manual) 2) การปรับปรุง/ต่อยอดระบบที่วางแผนไว้และระบุแล้ว 3) เปรียบเทียบต้นทุนและตารางเวลาจริงกับที่ประมาณไว้เดิม
User training	อบรมผู้ใช้ต่อเนื่อง (รวมพนักงานใหม่ และเมื่อระบบเปลี่ยน)
Help desk	จุดติดต่อกลางสำหรับงานบำรุงรักษา IT ทั้งหมด; เพิ่มผลิตภาพและการใช้ทรัพยากรสารสนเทศ; ใช้ automated support ได้หลายแบบ
Outsourcing issues	Offshore call center ลดค่าใช้จ่ายและปล่อยคนไปพัฒนาผลิตภัณฑ์; ปัจจัยสำคัญ: เวลารอสาย, ผลงานเจ้าหน้าที่, เครื่องมือสนับสนุนออนไลน์

5.7 การบำรุงรักษา (Maintenance) 4 ประเภท corrective, adaptive, perfective, preventive

ประเภท	ตามสไลด์	สัญญาณในโจทย์	ข้อควรรู้
Corrective แก้ไข	วินิจฉัยและ **แก้ข้อผิดพลาด** ของระบบที่ใช้งานอยู่	ระบบพัง บั๊ก ข้อมูลผิด	ตอบสนองตามลักษณะของ error; เลวสุด = ระบบล่ม; กลับมาใช้ได้แล้วต้องหาสาเหตุ วิเคราะห์ และออกแบบ **วิธีแก้ถาวร**
Adaptive ปรับเพิ่ม	**เพิ่มความสามารถ** (enhancements) ให้ระบบ และทำให้ใช้ง่ายขึ้น	เพิ่มฟีเจอร์/รายงาน ปรับให้ใช้ง่าย	ยากกว่าพัฒนาระบบใหม่ เพราะต้องทำภายใต้ข้อจำกัดของระบบเดิม
Perfective ปรับปรุงให้ดีขึ้น	เปลี่ยนระบบให้ **มีประสิทธิภาพ เชื่อถือได้ และดูแลรักษาง่ายขึ้น**	ทำให้เร็วขึ้น เสถียรขึ้น ปรับโครงสร้างโค้ด	คุ้มค่าที่สุดช่วง **กลางอายุ** ระบบ; โปรแกรมที่ต้องแก้มาก = ตัวเลือก **reengineering**; ยิ่งแก้มากยิ่งไม่มีประสิทธิภาพและดูแลยาก
Preventive ป้องกัน	วิเคราะห์จุดที่มีโอกาสเกิดปัญหา เพื่อป้องกันก่อนเกิด	ตรวจสอบเชิงรุก ป้องกันล่ม	**ฝ่าย IT** มักเป็นผู้เริ่ม; ผลคือผู้ใช้พอใจขึ้น downtime ลด และ **TCO** ลด; บางครั้งไม่ได้รับความสำคัญเท่าที่ควร

ใช้นิยามตามสไลด์ (Adaptive = เพิ่ม enhancement; Perfective = ประสิทธิภาพ/เชื่อถือได้/ดูแลง่าย). ตำราบางเล่มให้ adaptive = ปรับตามสภาพแวดล้อมที่เปลี่ยน ถ้าสับสนให้เขียนนิยามตามสไลด์กำกับ.

เกร็งข้อสอบ

โจทย์ถามแบบนี้	ตอบ / เลือกแบบนี้
ทดสอบระดับไหน	โปรแกรมเดียว = unit; 2 โปรแกรมขึ้นไปที่ส่งข้อมูลถึงกัน = integration; ทั้งระบบ = system
เสี่ยงสูงสุด/ประหยัดสุด	Direct cutover (เสี่ยงสูง ถูก)
เสี่ยงต่ำสุด/แพงสุด	Parallel operation (รัน 2 ระบบ)
ทดลองที่สาขาเดียวก่อน แล้วเปลี่ยนสาขาที่เหลือ	Pilot operation (ส่วนที่เหลือใช้ direct cutover)
เปลี่ยนทีละระบบย่อย/โมดูล	Phased operation; ต้องแบ่งโมดูลได้ ถูกกว่าและเสี่ยงน้อยกว่า parallel
ระบบใหม่ทำงานร่วมกับระบบเก่าไม่ได้	ใช้ parallel ไม่ได้
ซอฟต์แวร์สำเร็จรูป เช่น MS Office	Direct cutover
อบรมใคร	3 กลุ่ม: users, managers, IT staff
อบรมแบบไหน	vendor training, webinars/podcasts/tutorials, outside resources, in-house
ระบบล่ม/แก้ error	Corrective
เพิ่มฟีเจอร์/ทำให้ใช้ง่าย	Adaptive (ตามสไลด์)
ปรับให้เร็ว เสถียร ดูแลง่าย; ระบบอายุกลาง	Perfective (ถ้าแก้บ่อยมาก พิจารณา reengineering)
วิเคราะห์ปัญหาที่น่าจะเกิดแล้วป้องกัน, ลด TCO	Preventive (IT เป็นผู้เริ่ม)
ขั้นตอนแปลงข้อมูล	export จากระบบเก่า; ไม่ได้ก็เขียนโปรแกรมแปลง; ตรวจสอบความถูกต้อง
รายงานสรุปถึงผู้บริหารมีอะไร	เอกสารระบบฉบับสมบูรณ์, การปรับปรุงที่วางแผนไว้, เทียบต้นทุน/เวลาจริงกับประมาณการ

A ภาคผนวก Formulas, PV factor table

A.1 สูตรทั้งหมดในหน้าเดียว

เรื่อง	สูตร	ดูหน้า
เส้นทางวิกฤต	ความยาวเส้น = ΣD ของงานบนเส้น; ยาวสุด = วิกฤต	3
Forward / Backward	EF = ES + D; ES = max EF ก่อนหน้า; LS = LF - D; LF = min LS ถัดไป; Slack = LS - ES	3
Crash	m = Tn - Tc; k = (Cc - Cn)/(Tn - Tc); รวม = ก่อนหน้า + Σ(วัน×k)	4
Normalization	1NF ไม่มี repeating group; 2NF ไม่มี partial dependency; 3NF ไม่มี transitive dependency	10
Payback	n + (เงินที่ยังไม่คืน / กระแสเงินสดปีที่คืนทุน)	14
PV	$PV = V_t/(1+r)^t = V_t \times$ PV factor; ปี 0 factor = 1	15
Discounted payback	payback ที่ใช้ PV ของกระแสเงินสดแต่ละปี	15
NPV	$\Sigma (B_t - C_t)/(1+r)^t$ = PV รับ - PV จ่าย; > 0 ลงทุน	16
รายงาน	U = L - h - f; N = D + G×s + T; หน้า = [N/U]; เวลา = หน้า/ppm	20

A.2 ตาราง PV factor = $1/(1+r)^t$ อัตรา 5% ถึง 15%, ปีที่ 1 ถึง 10

ปี	5%	6%	7%	8%	9%	10%	11%	12%	13%	14%	15%
1	0.9524	0.9434	0.9346	0.9259	0.9174	0.9091	0.9009	0.8929	0.8850	0.8772	0.8696
2	0.9070	0.8900	0.8734	0.8573	0.8417	0.8264	0.8116	0.7972	0.7831	0.7695	0.7561
3	0.8638	0.8396	0.8163	0.7938	0.7722	0.7513	0.7312	0.7118	0.6931	0.6750	0.6575
4	0.8227	0.7921	0.7629	0.7350	0.7084	0.6830	0.6587	0.6355	0.6133	0.5921	0.5718
5	0.7835	0.7473	0.7130	0.6806	0.6499	0.6209	0.5935	0.5674	0.5428	0.5194	0.4972
6	0.7462	0.7050	0.6663	0.6302	0.5963	0.5645	0.5346	0.5066	0.4803	0.4556	0.4323
7	0.7107	0.6651	0.6227	0.5835	0.5470	0.5132	0.4817	0.4523	0.4251	0.3996	0.3759
8	0.6768	0.6274	0.5820	0.5403	0.5019	0.4665	0.4339	0.4039	0.3762	0.3506	0.3269
9	0.6446	0.5919	0.5439	0.5002	0.4604	0.4241	0.3909	0.3606	0.3329	0.3075	0.2843
10	0.6139	0.5584	0.5083	0.4632	0.4224	0.3855	0.3522	0.3220	0.2946	0.2697	0.2472

อ่าน: แถว = ปี t, คอลัมน์ = อัตรา r; ค่าที่ได้คูณกระแสเงินสดปีนั้นได้ PV. ตัวอย่าง: 12% ปีที่ 3 = 0.7118. ตารางที่แจกในห้องสอบเป็นของอาจารย์ (1% ถึง 50% และ 1 ถึง 50 งวด) ค่าเหมือนกัน 4 ตำแหน่ง.